Tugas Besar 1

IF3070 Dasar Inteligensi Artifisial 2026/2027

Kelompok 7 - bebas aja	Tsabita Khoirunnisa	18224006
	Wa Ode Amerta Lambelu J	18224034
	Melvin Irsyad Kuncoro	18224048

Daftar Isi

Daftar Isi **2**

Daftar Tabel **4**

Daftar Gambar **5**

Bab I: Deskripsi Persoalan **7**

Bab II: Pembahasan **20**

2.1 Fungsi Bersama (Shared Function) 20

2.1.1 File core.py 20

2.1.2 File neighbors.py 24

2.1.3 File problem.py 30

2.1.4 File visualisasi.py 36

2.1.5 File runner.py 42

2.1.6 File generator_testcase.py 48

2.2 Algoritma Hill Climbing 53

2.2.1 Hasil Percobaan Hill-Climbing with Sideways Move 56

2.2.2 Hasil tc_small.json 57

2.2.3 Hasil Hasil tc_medium.json 59

2.2.4 Hasil Hasil tc_large.json 62

2.2.5 Analisis 64

2.3 Algoritma Simulated Annealing 66

2.3.1 Hasil Percobaan Simulated Annealing 70

2.3.1.1 Percobaan Terhadap Perbedaan State Awal 70

2.3.1.2 Percobaan Terhadap Perbedaan Cooling Rate 77

2.3.1.3 Percobaan Keseluruhan 82

2.3.1.4 Analisis Keseluruhan 83

2.4 Algoritma Genetic 83

2.4.1 Hasil Percobaan 1: Jumlah Populasi sebagai Kontrol 91

2.4.2 Hasil Percobaan 2: Jumlah Iterasi sebagai Kontrol 98

2.4.3 Analisis 105
2.4.3.1 Pengaruh ukuran populasi terhadap hasil algoritma 105
2.4.3.2 Pengaruh jumlah generasi terhadap hasil algoritma 105
2.4.3.3 Waktu komputasi 106
2.4.3.4 Konsistensi hasil 106
2.5 Analisis Keseluruhan dan Perbandingan 107
Bab III: Kesimpulan dan Saran 113
3.1 Kesimpulan 113
a. Simulated Annealing 113
b. Hill Climbing 113
c. Genetic 114
3.2 Saran 114
Pembagian Tugas 115
Referensi 118
Lampiran Form Penggunaan Generative AI 120
Lampiran Lainnya 122

Daftar Tabel

Tabel 2.2.2.1 max_sideways = 0..... 57
Tabel 2.2.2.2 max_sideways = 100..... 58
Tabel 2.2.2.3 max_sideways = 200..... 58
Tabel 2.2.2.4 Rekap tc_small..... 59
Tabel 2.2.3.1 max_sideways = 0..... 60
Tabel 2.2.3.2 max_sideways = 100..... 61
Tabel 2.2.3.3 max_sideways = 200..... 61
Tabel 2.2.3.4 Rekap tc_small..... 62
Tabel 2.2.4.1 max_sideways = 0..... 62
Tabel 2.2.4.2 max_sideways = 100..... 63
Tabel 2.2.4.3 max_sideways = 200..... 63
Tabel 2.2.4.4 Rekap tc_small..... 64
Tabel 2.3.1.1.1 Hasil tc_small.json..... 71
Tabel 2.3.1.1.5 Hasil tc_medium.json..... 73
Tabel 2.3.1.1.9 Hasil tc_large.json..... 75
Tabel 2.3.1.2.1 Hasil c = 0.90..... 78
Tabel 2.3.1.2.2 Hasil c = 0.925..... 78
Tabel 2.3.1.2.5 Hasil c = 0.95..... 80
Tabel 2.3.1.3.1 Ringkasan tc_small.json..... 83
Tabel 2.3.1.3.2 Ringkasan tc_medium.json..... 83
Tabel 2.3.1.3.3 Ringkasan tc_large.json..... 83
Tabel 2.4.1.1 Hasil Percobaan 1: Jumlah Populasi sebagai Kontrol..... 93
Tabel 2.4.2.1 Hasil Percoban 2: Jumlah Iterasi Sebagai Kontrol..... 99

Daftar Gambar

Gambar 2.3.1.1.2 Hasil Run 1 tc_small.json.....72
Gambar 2.3.1.1.3 Hasil Run 2 tc_small.json.....73
Gambar 2.3.1.1.4 Hasil Run 3 tc_small.json.....73
Gambar 2.3.1.1.6 Hasil Run 1 tc_medium.json.....74
Gambar 2.3.1.1.7 Hasil Run 2 tc_medium.json.....74
Gambar 2.3.1.1.8 Hasil Run 3 tc_medium.json.....75
Gambar 2.3.1.1.10 Hasil Run 1 tc_large.json.....76
Gambar 2.3.1.1.11 Hasil Run 2 tc_large.json.....77
Gambar 2.3.1.1.12 Hasil Run 3 tc_large.json.....77
Gambar 2.3.1.2.3 Hasil Run 1 c = 0.925.....79
Gambar 2.3.1.2.4 Hasil Run 2 c = 0.925.....80
Gambar 2.3.1.2.4 Hasil Run 3 c = 0.925.....80
Gambar 2.3.1.2.6 Hasil Run 1 c = 0.95.....81
Gambar 2.3.1.2.7 Hasil Run 2 c = 0.95.....82
Gambar 2.3.1.2.8 Hasil Run 3 c = 0.95.....82
Gambar 2.4.1.1 Percobaan 1: Input tc_small.json iterasi 50 run 3/3.....95
Gambar 2.4.1.2 Percobaan 1: Input tc_small.json iterasi 100 run 2/2.....95
Gambar 2.4.1.3 Percobaan 1: Input tc_small.json iterasi 200 run 2/2.....96
Gambar 2.4.1.4 Percobaan 1: Input tc_medium.json iterasi 50 run 1/1.....96
Gambar 2.3.1.5 Percobaan 1: Input tc_medium.json iterasi 100 run 1/1.....97
Gambar 2.4.1.6 Percobaan 1: Input tc_medium.json iterasi 200 run 1/1.....97
Gambar 2.4.1.7 Percobaan 1: Input tc_large.json iterasi 50 run 1/1.....98
Gambar 2.4.1.8 Percobaan 1: Input tc_large.json iterasi 100 run 1/1.....98
Gambar 2.4.1.9 Percobaan 1: Input tc_large.json iterasi 200 run 1/1.....99
Gambar 2.4.1.1 Percobaan 2: Input tc_small.json populasi: 20 run 1/1.....102

Gambar 2.4.1.2 Percobaan 2: Input tc_small.json populasi: 50 run 2/2.....102
Gambar 2.4.1.3 Percobaan 2: Input tc_small.json populasi 100 run 2/2..... 103
Gambar 2.4.1.4 Percobaan 2: Input tc_medium.json populasi: 20 run 2/2.....103
Gambar 2.4.1.5 Percobaan 2: Input tc_medium.json populasi: 50 run 1/1.....104
Gambar 2.4.1.6 Percobaan 2: Input tc_medium.json populasi 100 run 3/3.....105
Gambar 2.4.1.7 Percobaan 1: Input tc_large.json iterasi 20 run 2/2..... 105
Gambar 2.4.1.8 Percobaan 1: Input tc_large.json iterasi 50run 1/1..... 105
Gambar 2.4.2.9 Percobaan 1: Input tc_large.json iterasi 100run 3/3.....106

Bab I: Deskripsi Persoalan

PT Intim Line merupakan perusahaan yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan menerima kontrak dari beberapa dealer sekaligus, sehingga pada setiap jadwal keberangkatan ada daftar kendaraan dengan ukuran, berat, tarif pengiriman, dan estimasi waktu kedatangan yang berbeda-beda. Tugas ini adalah membangun sistem pendukung keputusan yang bisa menentukan kendaraan mana yang diangkut, serta di mana dan dengan orientasi apa kendaraan itu diletakkan.

Geladak kapal dimodelkan sebagai grid dua dimensi berukuran W x L. Tiap kendaraan berupa persegi panjang dengan lebar dan panjang tertentu dalam satuan grid. Kami memilih beberapa kendaraan untuk dimuat, kemudian menentukan koordinat (x, y) dan orientasinya horizontal atau vertikal. Kendaraan yang tidak terpilih diasumsikan berada di luar kapal. Setiap kendaraan memiliki informasi *ShippingFee, Weight*, dan ETA. Sedangkan kapal memiliki informasi dimensi geladak dan *MaxCapacity*.

Tujuan dari sistem ini adalah memaksimalkan total *ShippingFee* dari kendaraan yang berada di atas kapal. Terdapat tiga batasan yang harus dipenuhi. Pertama, kendaraan tidak boleh saling bertindihan, sehingga satu sel geladak hanya boleh ditempati satu kendaraan. Kedua, seluruh sel yang ditempati harus berada dalam batas geladak. Ketiga, total berat seluruh kendaraan yang berada di kapal tidak boleh melebihi kapasitas maksimum kapal.

Dalam pengerjaan tugas ini, kami menggunakan pendekatan *local search*. Pencarian dimulai dari *state* awal yang diinisialisasi acak, lalu bergerak ke *state* tetangga lewat tiga jenis langkah yang diperbolehkan yaitu, menukar dua kendaraan (baik yang sama-sama di kapal maupun satu di dalam dan satu di luar), memindahkan satu kendaraan ke koordinat lain, dan mengubah orientasi kendaraan.

Tiga algoritma yang dibandingkan adalah *Hill-Climbing, Simulated Annealing,* dan *Genetic Algorithm*.

Untuk semua eksperimen di sistem kami diukur dengan ukuran state awal yang berbeda, dengan ketentuan state sebagai berikut:

tc_small.json	tc_medium.json	tc_small.json
{	{	{
"ship": {	"ship": {	"ship": {
"width": 5,	"width": 8,	"width": 14,
"length": 5,	"length": 12,	"length": 20,
"max_capacity": 49	"max_capacity": 109	"max_capacity": 398
},	},	},
"vehicles": [	"vehicles": [	"vehicles": [
{	{	{
"id": "V001",	"id": "V001",	"id": "V001",
"width": 2,	"width": 2,	"width": 5,
"length": 3,	"length": 2,	"length": 2,
"fee": 43,	"fee": 9,	"fee": 58,
"weight": 15,	"weight": 12,	"weight": 26,
"eta": 4	"eta": 2	"eta": 3
},	},	},
{	{	{
"id": "V002",	"id": "V002",	"id": "V002",
"width": 3,	"width": 1,	"width": 4,
"length": 3,	"length": 3,	"length": 6,
"fee": 28,	"fee": 20,	"fee": 175,
"weight": 23,	"weight": 3,	"weight": 50,
"eta": 4	"eta": 3	"eta": 5

```
},
{
 "id": "V003",
 "width": 3,
 "length": 3,
 "fee": 61,
 "weight": 11,
 "eta": 3
},
{
 "id": "V004",
 "width": 1,
 "length": 1,
 "fee": 5,
 "weight": 3,
 "eta": 6
},
{
 "id": "V005",
 "width": 1,
 "length": 3,
 "fee": 24,
 "weight": 9,
 "eta": 6
},
{
 "id": "V006",
 "width": 3,
 "length": 3,
```

```
},
{
 "id": "V003",
 "width": 1,
 "length": 3,
 "fee": 24,
 "weight": 4,
 "eta": 5
},
{
 "id": "V004",
 "width": 4,
 "length": 1,
 "fee": 22,
 "weight": 4,
 "eta": 3
},
{
 "id": "V005",
 "width": 3,
 "length": 4,
 "fee": 38,
 "weight": 32,
 "eta": 5
},
{
 "id": "V006",
 "width": 5,
 "length": 5,
```

```
},
{
 "id": "V003",
 "width": 2,
 "length": 6,
 "fee": 89,
 "weight": 35,
 "eta": 3
},
{
 "id": "V004",
 "width": 6,
 "length": 5,
 "fee": 147,
 "weight": 47,
 "eta": 6
},
{
 "id": "V005",
 "width": 6,
 "length": 1,
 "fee": 23,
 "weight": 10,
 "eta": 4
},
{
 "id": "V006",
 "width": 1,
 "length": 6,
```

```
    "fee": 53,
    "weight": 20,
    "eta": 7
   }
  ]
}
```

```
   "fee": 59,
   "weight": 41,
   "eta": 3
  },
  {
   "id": "V007",
   "width": 3,
   "length": 2,
   "fee": 46,
   "weight": 10,
   "eta": 7
  },
  {
   "id": "V008",
   "width": 5,
   "length": 2,
   "fee": 37,
   "weight": 15,
   "eta": 6
  },
  {
   "id": "V009",
   "width": 2,
   "length": 4,
   "fee": 18,
   "weight": 17,
   "eta": 1
  },
  {
```

```
   "fee": 34,
   "weight": 15,
   "eta": 4
  },
  {
   "id": "V007",
   "width": 3,
   "length": 3,
   "fee": 48,
   "weight": 21,
   "eta": 1
  },
  {
   "id": "V008",
   "width": 1,
   "length": 3,
   "fee": 24,
   "weight": 5,
   "eta": 2
  },
  {
   "id": "V009",
   "width": 1,
   "length": 6,
   "fee": 25,
   "weight": 14,
   "eta": 7
  },
  {
```

	"id": "V010",
	"width": 4,
	"length": 3,
	"fee": 60,
	"weight": 24,
	"eta": 4
	},
	{
	"id": "V011",
	"width": 4,
	"length": 2,
	"fee": 31,
	"weight": 14,
	"eta": 5
	},
	{
	"id": "V012",
	"width": 1,
	"length": 3,
	"fee": 16,
	"weight": 6,
	"eta": 5
	}
	]
	}
		"id": "V010",
		"width": 2,
		"length": 5,
		"fee": 36,
		"weight": 10,
		"eta": 5
		},
		{
		"id": "V011",
		"width": 6,
		"length": 3,
		"fee": 96,
		"weight": 32,
		"eta": 6
		},
		{
		"id": "V012",
		"width": 1,
		"length": 1,
		"fee": 4,
		"weight": 2,
		"eta": 6
		},
		{
		"id": "V013",
		"width": 5,
		"length": 1,
		"fee": 12,
		"weight": 6,

		"eta": 7 }, { "id": "V014", "width": 3, "length": 1, "fee": 7, "weight": 5, "eta": 2 }, { "id": "V015", "width": 2, "length": 6, "fee": 71, "weight": 23, "eta": 4 }, { "id": "V016", "width": 3, "length": 5, "fee": 79, "weight": 39, "eta": 3 }, { "id": "V017", "width": 4,

		"length": 1, "fee": 8, "weight": 7, "eta": 7 }, { "id": "V018", "width": 3, "length": 1, "fee": 18, "weight": 4, "eta": 5 }, { "id": "V019", "width": 6, "length": 4, "fee": 152, "weight": 66, "eta": 7 }, { "id": "V020", "width": 6, "length": 2, "fee": 90, "weight": 30, "eta": 4 },

		{ "id": "V021", "width": 1, "length": 6, "fee": 15, "weight": 13, "eta": 6 }, { "id": "V022", "width": 3, "length": 4, "fee": 44, "weight": 35, "eta": 4 }, { "id": "V023", "width": 3, "length": 4, "fee": 87, "weight": 35, "eta": 4 }, { "id": "V024", "width": 4, "length": 1, "fee": 26,

		"weight": 7, "eta": 3 }, { "id": "V025", "width": 2, "length": 1, "fee": 15, "weight": 6, "eta": 3 }, { "id": "V026", "width": 5, "length": 3, "fee": 112, "weight": 20, "eta": 6 }, { "id": "V027", "width": 3, "length": 4, "fee": 55, "weight": 31, "eta": 6 }, { "id": "V028",

		"width": 3, "length": 5, "fee": 89, "weight": 16, "eta": 6 }, { "id": "V029", "width": 6, "length": 6, "fee": 75, "weight": 71, "eta": 7 }, { "id": "V030", "width": 3, "length": 5, "fee": 37, "weight": 19, "eta": 7 }, { "id": "V031", "width": 2, "length": 2, "fee": 31, "weight": 5, "eta": 6

		}, { "id": "V032", "width": 1, "length": 5, "fee": 23, "weight": 10, "eta": 3 }, { "id": "V033", "width": 1, "length": 2, "fee": 9, "weight": 6, "eta": 7 }, { "id": "V034", "width": 2, "length": 4, "fee": 18, "weight": 8, "eta": 7 }, { "id": "V035", "width": 2, "length": 5,

		"fee": 34, "weight": 12, "eta": 3 }, { "id": "V036", "width": 1, "length": 6, "fee": 30, "weight": 10, "eta": 1 }, { "id": "V037", "width": 6, "length": 1, "fee": 41, "weight": 12, "eta": 3 }, { "id": "V038", "width": 5, "length": 3, "fee": 59, "weight": 19, "eta": 3 }, {

		"id": "V039",
		"width": 1,
		"length": 3,
		"fee": 13,
		"weight": 8,
		"eta": 3
		},
		{
		"id": "V040",
		"width": 2,
		"length": 1,
		"fee": 10,
		"weight": 5,
		"eta": 3
		}
		]
		}

Bab II: Pembahasan

2.1 Fungsi Bersama *(Shared Function)*

2.1.1 File core.py

File core.py bertanggung jawab atas fungsi-fungsi dasar untuk menilai sebuah state, yaitu menghitung nilai objective dan memeriksa apakah state memenuhi seluruh constraint.

Terdapat 6 fungsi pada file ini:

```
def footprint(vehicle, placement):
    width = vehicle["width"]
    length = vehicle["length"]
    if (placement["orientation"] == "H"):
        width_x = width
        length_y = length
    elif (placement["orientation"] == "V"):
        width_x = length
        length_y = width
    else:
        orientation = placement["orientation"]
            raise ValueError(f"Tidak ada orientasi: {orientation!r}, Orientasi yang valid hanya H/V")
    x_akhir = placement["x"] + width_x
    y_akhir = placement["y"] + length_y
    x_awal = placement["x"]
    y_awal = placement["y"]
    return (x_awal, y_awal, x_akhir, y_akhir)
```

Fungsi footprint mengubah sebuah kendaraan beserta penempatannya menjadi kotak persegi panjang pada grid, dalam bentuk tuple (x_awal, y_awal, x_akhir, y_akhir). Koordinat x dan y pada penempatan dipakai sebagai sudut awal kotak. Jika orientasi "H", lebar kendaraan mengikuti sumbu x dan panjangnya mengikuti sumbu y, sedangkan jika "V" keduanya ditukar. Koordinat akhir dihitung dengan menambahkan ukuran tersebut ke koordinat awal dan bersifat eksklusif. Contohnya, kendaraan selebar 3 yang dimulai dari x=2 menempati sel 2, 3, dan 4 dengan x_akhir = 5. Jika orientasi selain "H" atau "V", fungsi melempar ValueError agar kesalahan data tidak menghasilkan kotak yang salah tanpa disadari.

```
def in_bound(box, ship):
    return (box[0] >= 0 and
            box[1] >= 0 and
            box[2] <= ship["width"] and
            box[3] <= ship["length"])
```

Fungsi in_bound memeriksa apakah sebuah kotak berada sepenuhnya di dalam geladak, sesuai constraint kendaraan harus berada dalam batas geladak. Koordinat awal tidak boleh negatif, dan koordinat akhir tidak boleh melebihi lebar geladak (sumbu x) maupun panjang geladak (sumbu y). Karena koordinat akhir eksklusif, perbandingannya memakai <=, sehingga kendaraan yang tepat menempel pada tepi geladak tetap dianggap berada di dalam.

```
def overlap(box1, box2):
    return not (box1[2] <= box2[0] or
                box2[2] <= box1[0] or
                box1[3] <= box2[1] or
```

```
            box2[3] <= box1[1])

# Fungsi overlap memeriksa apakah dua kotak saling bertumpuk dengan membalik pertanyaannya. Dua kotak dikatakan tidak bertumpuk apabila salah satunya sepenuhnya berada di sebelah kiri, kanan, atas, atau bawah kotak yang lain. Jika tidak ada satu pun dari empat kondisi tersebut yang terpenuhi, berarti ada sel yang dipakai bersama dan fungsi mengembalikan True. Karena memakai <=, dua kendaraan yang hanya bersebelahan di tepi dianggap tidak bertumpuk.
```

```
def total_weight(state, problem):
    total = 0
    for (i,vehicle) in enumerate(problem["vehicles"]):
        placement = state[i]
        if placement["location"] == 0:
            total += problem["vehicles"][i]["weight"]
    return total

# Fungsi total_weight menjumlahkan berat seluruh kendaraan yang berada di dalam kapal (location == 0). Kendaraan yang berada di luar kapal tidak dihitung. Hasilnya dibandingkan dengan max_capacity pada fungsi is_valid.
```

```
def is_valid(state, problem):
    if total_weight(state, problem) > problem["ship"]["max_capacity"]:
        return False
    for (i,vehicle) in enumerate(problem["vehicles"]):
        placement = state[i]
        if placement["location"] != 0:
```

```
            continue
        box1 = footprint(vehicle, placement)
        if not in_bound(box1, problem["ship"]):
            return False
        for j in range(i+1, len(problem["vehicles"])):
            placement2 = state[j]
            if placement2["location"] != 0:
                continue
            vehicle2 = problem["vehicles"][j]
            box2 = footprint(vehicle2, placement2)
            if overlap(box1, box2):
                return False
    return True


# Fungsi is_valid menggabungkan ketiga constraint pada spesifikasi, yaitu kapasitas, batas geladak, dan no overlapping. Pertama, total berat dibandingkan dengan kapasitas kapal, dan jika melebihi, fungsi langsung mengembalikan False tanpa memeriksa yang lain. Selanjutnya, untuk setiap kendaraan yang berada di kapal, dibuat kotaknya dengan footprint, diperiksa apakah masih dalam batas geladak, lalu dibandingkan dengan kendaraan-kendaraan setelahnya (indeks j mulai dari i+1) untuk mendeteksi overlap. Dengan hanya membandingkan indeks yang lebih besar, setiap pasangan kendaraan diperiksa satu kali. Kendaraan dengan location != 0 dilewati pada kedua loop. Kompleksitas fungsi ini adalah O(n²) terhadap jumlah kendaraan di kapal. Hal ini berpengaruh pada biaya pembangkitan seluruh neighbor, terutama untuk Steepest Ascent Hill-Climbing yang harus memeriksa validitas setiap neighbor.
```

```
def objective(state, problem):
    total = 0
    for (i,vehicle) in enumerate(problem["vehicles"]):
        placement = state[i]
        if placement["location"] == 0:
            total += problem["vehicles"][i]["fee"]
    return total

# Fungsi objective menjumlahkan fee seluruh kendaraan yang berada di dalam kapal, sesuai tujuan pada spesifikasi, yaitu memaksimalkan total ShippingFee. Fungsi ini tidak memeriksa validitas state dan mengandalkan is_valid untuk menjamin state yang dinilai sudah sah. Oleh karena itu, state awal yang dibangkitkan secara acak juga harus dipastikan valid sebelum dihitung nilainya.
```

2.1.2 File neighbors.py

File neighbors.py bertanggung jawab atas fungsi-fungsi yang menghasilkan tetangga (*neighbors*).

Terdapat 11 fungsi pada file ini, di antaranya yaitu:

```
def copy_state(state):
    return [dict(v) for v in state]

# fungsi ini berfungsi untuk membuat salinan baru dari suatu state
```

```
def update_state(state, updates):
    new_state = copy_state(state)
    for i, v in updates.items():
```

```
        new_state[i] = v
    return new_state

# fungsi ini mengembalikan state baru yang sudah perbarui oleh updates. Updates merupakan list of dict dengan key: value, key berupa indeks kendaraan dalam value, kemudian value adalah dict dari peletakan kendaraan. Indeks berfungsi agar perubahan hanya dilakukan pada kendaraan yang mengalami perubahan peletakan.
```

Kemudian, untuk menghasilkan neighbor maka perlu diketahui gerakan apa yang mungkin dilakukan pada 1 iterasi. Berdasarkan spesifikasi tugas besar dan diskusi kelompok, berikut merupakan daftar gerakan:

a. Berpindah koordinat (dengan orientasi tetap)
b. Berputar orientasi
c. Memasukkan kendaraan
d. Mengeluarkan kendaraan
e. Menukar 2 kendaraan dengan keduanya berada dalam kapal
f. Menukar 2 kendaraan dengan salah satunya terdapat di dalam kapal dan yang lain terdapat di luar kapal

Berdasarkan 6 gerakan tersebut dibuat 6 fungsi bantuan

```
def shift(state, problem):
    width = problem["ship"]["width"]
    length = problem["ship"]["length"]
    res = []

    for i, v in enumerate(state):
```

```
    if v["location"] != 0:
      continue
    for x in range(width):
      for y in range(length):
        if x == v["x"] and y == v["y"]:
          continue
        res.append(update_state(state, {i: {**v, "x": x, "y": y}}))

  return res

# fungsi untuk jenis gerakan 1, dilakukan pengecekan awal apakah kendaraan berada di dalam kapal atau tidak. Jika tidak, lanjut ke iterasi selanjutnya. Kemudian, dilakukan iterasi terhadap kombinasi untuk setiap koordinat berdasarkan dimensi kapal. Jika koordinat sama dengan koordinat kendaraan saat ini, continue. Jika lolos semua pengecekan,  menambahkan state tersebut pada list of state. Setelah iterasi selesai mengembalikan res (list of state).
```

```
def rotate(state, problem):
  res = []

  for i, v in enumerate(state):
    if v["location"] != 0:
      continue
    if v["orientation"] == "H":
      new_o = "V"
    else:
      new_o = "H"
    res.append(update_state(state, {i: {**v, "orientation": new_o}}))
```

```
    return res
```

fungsi untuk jenis gerakan 2, dilakukan pengecekan awal apakah kendaraan berada di dalam kapal atau tidak. Jika tidak, lanjut ke iterasi selanjutnya. Kemudian mengupdate orientasi baru dan menambahkannya ke *res*. Setelah iterasi berakhir, fungsi ini mengembalikan *res*.

```
def enter(state, problem):
    width = problem["ship"]["width"]
    length = problem["ship"]["length"]
    res = []

    for i, v in enumerate(state):
        if v["location"] != -1:
            continue
        for x in range(width):
            for y in range(length):
                for o in ("H", "V"):
                        res.append(update_state(state, {i: {"location": 0, "x": x, "y": y, "orientation": o}}))

    return res
```

fungsi untuk jenis gerakan 3, dilakukan pengecekan awal apakah kendaraan berada di dalam kapal atau tidak. Jika iya, lanjut ke iterasi selanjutnya. Jika tidak, maka menghasilkan state dengan kombinasi koordinat dan 2 orientasi.

```
def outside():
    return {"location": -1, "x": 0, "y": 0, "orientation": "H"}

def leave(state, problem):
    res = []

    for i, v in enumerate(state):
        if v["location"] != 0:
            continue
        res.append(update_state(state, {i: outside()}))

    return res
```

fungsi untuk jenis gerakan 4, dilakukan pengecekan awal apakah kendaraan berada di dalam kapal atau tidak. Jika tidak, lanjut ke iterasi selanjutnya. Jika iya, maka kapal dikeluarkan. Fungsi *outside* digunakan untuk memudahkan penulisan peletakan kendaraan yang berada di luar kapal.

```
def swap_2in(state, problem):
    in_ship = [i for i, v in enumerate(state) if v["location"] == 0]
    res = []

    for i, j in combinations(in_ship, 2):
        k, l = state[i], state[j]
        if k["x"] == l["x"] and k["y"] == l["y"]:
            continue
        res.append(update_state(state, {i: {**k, "x": l["x"], "y": l["y"]}, j:
{**l, "x": k["x"], "y": k["y"]}}))
```

```
    return res

# fungsi untuk jenis gerakan 5, mengambil seluruh kendaraan yang sudah berada di kapal, kemudian mencari seluruh kombinasi 2 kendaraan. Jika kombinasi berisikan kapal yang sama, maka lanjut ke iterasi selanjutnya. Jika tidak, maka akan menambahkan state baru dengan menukar peletakan 2 kendaraan tersebut tetapi tetap memiliki orientasi yang sama.
```

```
def swap_inout(state, problem):
    in_ship = [i for i, v in enumerate(state) if v["location"] == 0]
    out_ship = [j for j, v in enumerate(state) if v["location"] == -1]
    res = []

    for i in in_ship:
        for j in out_ship:
            for o in ("H", "V"):
                res.append(update_state(state, {i: outside(), j: {"location": 0, "x": state[i]["x"], "y": state[i]["y"], "orientation": o}}))

    return res

# fungsi untuk jenis gerakan 6, mengambil seluruh kendaraan yang ada di dalam dan luar kapal. Kemudian, mencari kombinasi antar keduanya. Karena, kendaraan yang di luar kapal tidak memiliki orientasi yang bermakna secara langsung, ketika masuk dapat memiliki orientasi horizontal ataupun vertikal.
```

Setelah semua fungsi pembantu sudah ada, selanjutnya adalah menyusun fungsi *all_neighbors* dan *random_neighbor*.

```
ALL_MOVE = [shift, rotate, enter, leave, swap_2in, swap_inout]

def all_neighbors(state, problem):
    res = []

    for move in ALL_MOVE:
        for s in move(state, problem):
            if is_valid(s, problem):
                res.append(s)

    return res

# fungsi untuk mencari semua neighbor valid, menjalankan seluruh fungsi gerakan kemudian menambahkan ke res jika valid.
```

```
def random_neighbor(state, problem, rng):
    neighbors = all_neighbors(state, problem)
    if not neighbors:
        return None
    return rng.choice(neighbors)

# fungsi mencari neighbor acak, dengan memilih satu neighbor dari all_neighbors() secara acak
```

2.1.3 File problem.py

File problem.py bertanggung jawab untuk mengubah input JSON menjadi objek dictionary yang bisa diolah oleh Python, menggenerate random state, dan menyimpan hasil akhir dalam json.

```
def check_int_positive(value, field_name):
    if isinstance(value, bool) or not (isinstance(value, int) and value > 0):
        raise ValueError(f"{field_name} harus bulat positif")

# fungsi helper untuk mengecek apakah input angka merupakan bilangan bulan positif. Tugas ini dibuat fungsi tersendiri karena banyak dipakai saat validasi komponen-komponen kapal dan kendaraan (width kapal, length kapal, max_capacity kapal, width kendaraan, lentgh kendaraan, fee kendaraan, weight kendaraan, dan ETA kendaraan)
```

```
def load_problem(path):
    with open(path, "r", encoding="utf-8") as file:
        problem = json.load(file)

    # validasi di json harus ada ship sama vehicles
    if "ship" not in problem or "vehicles" not in problem:
        raise ValueError("file harus memiliki ship dan vehicles")

    # validasi w, l, max kapasitas kapal harus int positif
    ship = problem["ship"]
    check_int_positive(ship.get("width"), "lebar kapal")
    check_int_positive(ship.get("length"), "panjang kapal")
```

```
    check_int_positive(ship.get("max_capacity"), "kapasitas maksimal kapal")

    # validasi komponen2 di kendaraan
    seen_id = set()
    for vehicle in problem["vehicles"]:
        vehicle_id = vehicle.get("id")
    if not isinstance(vehicle_id, str):
        raise ValueError("ID kendaraan harus string")
    if vehicle_id in seen_id:
        raise ValueError(f"ID {vehicle_id} duplikat")
    seen_id.add(vehicle_id)

            check_int_positive(vehicle.get("width"), f"lebar kendaraan {vehicle_id}")
        check_int_positive(vehicle.get("length"), f"panjang kendaraan {vehicle_id}")
                check_int_positive(vehicle.get("fee"), f"fee kendaraan {vehicle_id}")
            check_int_positive(vehicle.get("weight"), f"berat kendaraan {vehicle_id}")
                check_int_positive(vehicle.get("eta"), f"eta kendaraan {vehicle_id}")

    return problem

# fungsi ini menerima dan membaca file JSON input dan memvalidasi apakah file input memiliki parameter-parameter yang benar (seperti contoh  JSON input harus setidaknya memiliki kapal dan kendaraan,
```

bilangan-bilangan yang harus bulat positif, tipe data ID harus string, dan tidak boleh ada duplikat.

```
def random_state(problem, rng, max_attempts=10):
    vehicles = problem["vehicles"]
    ship = problem["ship"]
    total = len(vehicles)

    state = []
    for _ in range (total):
        state.append({"location": -1, "x": 0, "y": 0, "orientation": "H"})

    #acak urutan kendaraan yg mau dimasukin
    order = list(range(total))
    rng.shuffle(order)

    #masukin kendaraan ke kapal
    for index in order:
        v = vehicles[index]

        # loop max attempts buat orientasi
        for _ in range (max_attempts):
            orientation = rng.choice(["H", "V"])

            if orientation == "H":
                w2 = v["width"]
                l2 = v["length"]
            else:
                w2 = v["length"]
```

```
            l2 = v["width"]

        limit_x = ship["width"] - w2
        limit_y = ship["length"] - l2

        if limit_x < 0 or limit_y < 0:
            continue

            state[index] = {"location": 0, "x": rng.randint(0, limit_x), "y":
rng.randint(0, limit_y), "orientation": orientation}

        if is_valid(state, problem) == True:
            break
        else:
            state[index] = {"location": -1, "x": 0, "y": 0, "orientation": "H"}
    return state

# Fungsi ini dipakai buat bikin state awal secara random untuk
tiap-tiap kendaraan. Sstate punya variabel lokasi (0 artinya di dalam
kapal, -1 artinya di luar kapal), titik koordinat sumbu x, sumbu y, dan
orientasi (vertikal atau horizontal). Awalnya, semua kendaraan
diasumsikan antre di luar kapal (lokasi -1). Kemudian sistem akan
memasukkan kendaraan satu per satu. Urutan masuknya sengaja
diacak supaya hasilnya lebih beragam dan nggak cuma
memprioritaskan mobil-mobil yang ada di urutan awal indeks. Saat
sebuah kendaraan giliran dimasukan, hal pertama yang diacak adalah
orientasinya. (Proses pencarian ini dibatasi oleh max_attempts biar
programnya tidak stuck atau infinite loop kalau kapalnya sudah
penuh). Setelah orientasi ditentukan, sistem akan mengitung batas
```

maksimal rentang x dan y yang muat di kapal. Kalau ternyata ukurannya lebih besar dari sisa ruang (hasilnya minus), sistem akan langsung skip dan mecoboa randomisasi posisi lagi. Tapi kalau muat, koordinat x dan y-nya akan diacak. Begitu posisinya didappat, sistem akan cek fungsi is_valid untuk memastikan mobilnya tidak tumpang tindih sama kendaraan lain dan total beratnya masih bisa ditrampung kapal. Kalau posisinya valid, mobil itu ditaruh di dalam kapal (lokasinya jadi 0) dan proses lanjut ke mobil berikutnya. Tapi kalau ternyata melanggar aturan, mobil tersebut akan ditarik lagi ke luar kapal (kembali ke -1) dan sistem akan terus menrandomisasi ulang titik posisinya sampai jatah max_attempts habis.

```
def config_to_text(config):
    if not config:
        return "default"
                return "_".join(f"{key}{value}" for key, value in sorted(config.items()))

# Fungsi helper untuk penamaan file.
```

```
def save_output(result, output_directory="output"):
        name = f"{result ['algorithm']}_{config_to_text(result.get('config', {}))}_run{result['run']}"
    json_path = os.path.join(output_directory, name + ".json")

    with open(json_path, 'w', encoding="utf-8") as file:
        json.dump(result, file, indent = 2, ensure_ascii = False)
    return json_path
```

```
# Fungsi untuk menyimpan hasil run, state awal, state akhir, durasi sebagai output algoritma.
```

2.1.4 File visualisasi.py

File ini bertanggung jawab atas seluruh visualisasi hasil run: menggambar grid geladak (state awal dan akhir), kurva objective terhadap iterasi, dan ringkasan informasi run. Library yang dipakai adalah matplotlib, dan fungsi footprint serta objective diimpor dari core.py supaya ukuran kotak kendaraan dan nilai state dihitung dengan cara yang sama seperti di algoritma.

```
def draw_state(ax, state, problem, judul):
    lebar_kapal = problem["ship"]["width"]
    panjang_kapal = problem["ship"]["length"]

    for i, kendaraan in enumerate(state):
        if kendaraan["location"] != 0:
            continue

        x1, y1, x2, y2 = footprint(problem["vehicles"][i], kendaraan)
        lebar = x2 - x1
        panjang = y2 - y1

        kotak = Rectangle((x1, y1), lebar, panjang,
                          facecolor="skyblue", edgecolor="SteelBlue",
linewidth=1)
        ax.add_patch(kotak)
        ax.text(x1 + lebar / 2, y1 + panjang / 2,
problem["vehicles"][i]["id"],
```

```
        ha="center", va="center", fontsize=10)


    ax.set_xlim(0, lebar_kapal)
    ax.set_ylim(0, panjang_kapal)
    ax.set_yticks(range(panjang_kapal + 1))
    ax.set_xticks(range(lebar_kapal + 1))
    ax.set_aspect("equal")
    ax.invert_yaxis()
    ax.set_title(judul)


# Fungsi ini menggambar geladak kapal pada sebuah sumbu (ax). Untuk tiap kendaraan yang berada di kapal (location == 0), kotaknya dihitung dengan footprint, lalu digambar sebagai persegi panjang biru muda dengan id kendaraan di tengahnya. Kendaraan di luar kapal dilewati. Setelah semua kendaraan tergambar, batas sumbu diatur sesuai ukuran geladak, tick dibuat per sel grid, dan set_aspect("equal") menjaga agar sel tetap berbentuk persegi. Sumbu y dibalik (invert_yaxis) supaya titik (0, 0) ada di kiri atas, seperti cara membaca grid pada umumnya.
```

```
def draw_trend(ax, history, judul, y_label):
    ax.plot(history)
    ax.set_title(judul)
    ax.set_xlabel("iterasi")
    ax.set_ylabel(y_label)
    ax.grid(True)


# Fungsi sederhana untuk memplot satu deret nilai terhadap iterasi, misalnya riwayat objective milik Hill-Climbing atau SA. Sumbu x
```

otomatis berupa indeks (nomor iterasi) dan sumbu y diberi label sesuai parameter y_label.

```
def draw_state_lengkap(ax, state, problem, judul):
    nilai = objective(state, problem)
    berat = 0
    for i, s in enumerate(state):
        if s["location"] == 0:
            berat += problem["vehicles"][i]["weight"]
    kapasitas = problem["ship"]["max_capacity"]

    draw_state(ax, state, problem, f"{judul}\nnilai {nilai}, berat {berat}/{kapasitas}")

    di_luar = []
    for i, s in enumerate(state):
        if s["location"] == -1:
            di_luar.append(problem["vehicles"][i]["id"])

    if len(di_luar) == 0:
        teks = "Di luar kapal: -"
    else:
        teks = f"Di luar kapal ({len(di_luar)}): " + ", ".join(di_luar[:MAKS_ID])
        if len(di_luar) > MAKS_ID:
            teks += f", ... (+{len(di_luar) - MAKS_ID})"

    ax.set_xlabel(teks, fontsize=8)
```

Fungsi ini membungkus draw_state dengan informasi tambahan yang diminta spesifikasi: kondisi kendaraan di dalam dan di luar kapal, serta value state. Judul grid diisi nilai objective dan berat terhadap kapasitas (misalnya “berat 120/150”). Berat dihitung langsung di sini dengan menjumlahkan kendaraan yang ada di kapal. Selanjutnya id semua kendaraan yang berada di luar kapal dikumpulkan dan ditulis di bawah grid. Jika jumlahnya lebih dari MAKS_ID, sisanya diringkas menjadi “... (+n)” agar label tidak melebar.

def draw_trend_multi(ax, series, judul, y_label, x_label="iterasi"):
  for label, history in series.items():
  ax.plot(history, label=label)
  ax.set_title(judul)
  ax.set_xlabel(x_label)
  ax.set_ylabel(y_label)
  ax.grid(True)
  ax.legend()
 	
# Versi draw_trend yang bisa menggambar beberapa kurva sekaligus dalam satu plot, lengkap dengan legend. Parameter series berupa dictionary berisi nama kurva dan datanya. Fungsi ini dipakai untuk Genetic Algorithm, yang harus menampilkan objective maksimum dan rata-rata populasi bersamaan. Label sumbu x bisa diganti, misalnya menjadi “generasi”. 	

def draw_info(ax, lines):   ax.axis("off")   ax.text(0.0, 1.0, "\n".join(lines), va="top", ha="left",   family="monospace", fontsize=9, transform=ax.transAxes)

```
def info_lines(result):
    lines = [f"Algoritma   : {result['algorithm']}"]
    for key, value in result.get("config", {}).items():
        lines.append(f"{key}: {value}")
    lines.append("")
    lines.append(f"Nilai akhir: {result['final_value']}")
    lines.append(f"Durasi: {result['duration_s']:.3f} detik")
    lines.append(f"Iterasi: {result.get('iterations')}")
    lines.append(f"Evaluasi: {result.get('evaluations')}")
    lines.append(f"Seed/run: {result.get('seed')} / {result.get('run')}")
    return lines
```

info_lines menyusun ringkasan satu run menjadi daftar baris teks: nama algoritma, konfigurasi parameter (misalnya populasi atau temperature awal), nilai akhir, durasi, jumlah iterasi dan evaluasi, serta seed dan nomor run. Nilai yang tidak ada diambil dengan .get, sehingga fungsi ini aman dipakai untuk semua algoritma meski field-nya berbeda. draw_info lalu menampilkan baris-baris itu sebagai teks pada sumbu kosong (axis("off")) dengan font monospace supaya rapi sejajar.

```
def draw_result(result, problem, save_path=None, show=False):

    fig, axes = plt.subplots(2, 2, figsize=(11, 8.5))
        draw_state_lengkap(axes[0, 0], result["initial_state"], problem, "State awal")
        draw_state_lengkap(axes[0, 1], result["final_state"], problem, "State akhir")
```

```
  if "history_max" in result:
    draw_trend_multi(
      axes[1, 0],
                  {"maksimum": result["history_max"], "rata-rata":
result["history_mean"]},
                   "Objective populasi per generasi", "Objective",
x_label="generasi",
    )
    draw_info(axes[1, 1], info_lines(result))
  else:
      draw_trend(axes[1, 0], result["history"], "Objective per iterasi",
"Objective")
    probs = result.get("extra", {}).get("accept_prob_history")
    if probs:
            draw_trend(axes[1, 1], probs, "Probabilitas per iterasi",
"e^(delta/T)")
      stuck = result.get("extra", {}).get("count_stuck")
      if stuck is not None:
            axes[1, 1].set_xlabel(f"iterasi (stuck di local optimum:
{stuck}x)")
    else:
      draw_info(axes[1, 1], info_lines(result))

  fig.suptitle(f"{result['algorithm']} - run {result.get('run', '?')}")
  fig.tight_layout(rect=[0, 0, 1, 0.96])
  if save_path:
    fig.savefig(save_path, dpi=150)
  if show:
```

```
        plt.show()
    plt.close(fig)


# Fungsi draw_result merangkai hasil satu run menjadi satu gambar 2×2 (11 × 8,5 inci) dengan judul berisi nama algoritma dan nomor run. Baris atas selalu menampilkan state awal dan state akhir lewat draw_state_lengkap, yang memuat grid, nilai objective, berat muatan terhadap kapasitas, serta daftar kendaraan di luar kapal. Baris bawah ditentukan oleh kunci yang ada di result, bukan oleh nama algoritma: jika ada history_max (GA), panel kiri bawah menampilkan kurva objective maksimum dan rata-rata populasi per generasi dan panel kanan bawah berisi ringkasan info; jika tidak, panel kiri bawah menampilkan kurva objective per iterasi, sedangkan panel kanan bawah menampilkan plot probabilitas e^(ΔE/T) bila accept_prob_history tersedia (SA), lengkap dengan jumlah stuck di local optimum pada label sumbu x bila count_stuck ada, atau ringkasan info bila tidak tersedia (Hill-Climbing). Gambar kemudian disimpan ke save_path pada 150 dpi atau ditampilkan dengan show=True, lalu figure ditutup dengan plt.close agar memori tidak menumpuk saat banyak run dijalankan berurutan.
```

2.1.5 File runner.py

File ini beranggung jawab untuk menjalankan seluruh algoritma beserta argumen-argumen yang diperlukannya

```
def run_sa(problem, rng, temperature=100, cooling_rate=0.95, max_iterations=1000):
```

```
    state = random_state(problem, rng)
    initial_state = copy.deepcopy(state)
    hasil = SA(state, problem, temperature, cooling_rate,
max_iterations, rng)
    _, final_state, final_value, history, duration, prob_history, countt,
stuckk = hasil

    return {
        "algorithm": "Simulated Annealing",
        "config": {
            "temperature": temperature,
            "cooling_rate": cooling_rate,
            "max_iterations": max_iterations,
        },
        "initial_state": initial_state,
        "final_state": final_state,
        "final_value": final_value,
        "history": history,
        "duration_s": duration,
        "iterations": len(history),
        "evaluations": len(history) + 1,
        "extra": {"accept_prob_history": prob_history, "count_stuck":
countt},
    }

# fungsi untuk menjalankan algoritma dengan menerima parameter
sesuai dengan fungsi SA(), 100. 0.95, dan 1000 adalah angka default
yang dapat dioverwrite. Hasil dikembalikan agar dapat disimpan pada
file output.
```

```
def run_hc(problem, rng, max_sideways=100):
    hasil = hill_climbing_sideways(problem, rng, max_sideways)
    mulai, akhir, nilai_akhir, history, durasi, iterasi, evaluasi, _,
total_sideways, alasan = hasil

    return {
        "algorithm": "Hill-Climbing Sideways",
        "config": {"max_sideways": max_sideways},
        "initial_state": mulai,
        "final_state": akhir,
        "final_value": nilai_akhir,
        "history": history,
        "duration_s": durasi,
        "iterations": iterasi,
        "evaluations": evaluasi,
        "extra": {
            "total_sideways": total_sideways,
            "alasan": alasan,
        },
    }
```

fungsi untuk menjalankan algoritma hill climbing dengan menerima parameter sesuai fungsi hill_climbing_sideways. 100 adalah angka default yang dapat dioverwrite. Hasil dikembalikan agar dapat disimpan pada file output.

```
EXPERIMENTS = {
    "GA_iteration": {
        "fn": genetic_algorithm,
```

```
    "configs": [{"population_size": 50, "max_generations": g} for g in
(50, 100, 200)],
  },
  "GA_population": {
    "fn": genetic_algorithm,
    "configs": [{"population_size": p, "max_generations": 100} for p in
(20, 50, 100)],
  },
  "SA": {
    "fn": run_sa,
    "configs": [{"temperature": 100, "cooling_rate": a,
"max_iterations": 500} for a in (0.9, 0.925, 0.95)],
  },
  "HC": {
    "fn": run_hc,
    "configs": [{"max_sideways": s} for s in (0, 100, 200)],
  }
}
```

Konstanta seluruh konfigurasi yang digunakan untuk eksperimen. GA_iteration merupakan eksperimen dengan mengubah nilai dari max_generations, GA_population merupakan eksperimen dengan mengubah nilai population_size, SA merupakan eksperimen simulated annealing dengan mengubah nilai dari cooling_rate, dan HC merupakan eksperimen dengan mengubah nilai max_sideways

```
def run_experiments(testcase_path, names, out_root, seeds,
make_figures):
  problem = load_problem(str(testcase_path))
```

```
    out_dir = os.path.join(out_root, Path(testcase_path).stem)
    os.makedirs(out_dir, exist_ok=True)

    rows = []
    done = set()
    for exp_name in names:
        exp = EXPERIMENTS[exp_name]
        for config in exp["configs"]:
            for run_number, seed in enumerate(seeds, start=1):
                key = (exp["fn"].__name__, tuple(sorted(config.items())),
run_number)
                if key in done:
                    continue
                done.add(key)

                rng = random.Random(seed)
                result = exp["fn"](problem, rng, **config)
                result["run"] = run_number
                result["seed"] = seed

                save_output(result, out_dir)
                config_text = config_to_text(result.get("config", {}))
                if make_figures:
                    base =
f"{result['algorithm']}_{config_text}_run{run_number}"
                    draw_result(result, problem, os.path.join(out_dir, base +
"_ringkasan.png"))

                rows.append({
```

```
                "experiment": exp_name,
                "algorithm": result["algorithm"],
                "config": config_text,
                "run": run_number,
                "seed": seed,
                "final_value": result["final_value"],
                "duration_s": round(result["duration_s"], 4),
                "iterations": result.get("iterations"),
                "evaluations": result.get("evaluations"),
            })
            print(f"[{exp_name}] {config_text} run{run_number} (seed
{seed}): "
                  f"nilai={result['final_value']}, {result['duration_s']:.2f}
detik")

    summarize(rows, out_dir)
    print(f"\nHasil ada di {out_dir}")
```

Fungsi utama eksperimen. Menerima parameter problem dari path test case dan membaca json. Rows akan digunakan untuk menyimpan ringkasan setiap run, done akan digunakan sebagai set agar 2 konfigurasi kembar tidak dijalankan 2 kali. Loop bersarang menghasilkan jumlah run sebanyak exp_name (GA_iteration/population, SA, HC) * config * seed. Seed yang digunakan pada kode kami terdapat 3, yaitu [1, 2, 3]. Kemudian dilanjutkan dengan menjalankan masing-masing run dan menyimpannya.

```
def summarize(rows, out_dir):
```

```
    "Rata-rata, terbaik, terburuk, dan simpangan baku per konfigurasi."
    groups = {}
    for r in rows:
        groups.setdefault((r["algorithm"], r["config"]), []).append(r)

    agg_rows = []
    for (algo, cfg), items in groups.items():
        values = [r["final_value"] for r in items]
        durations = [r["duration_s"] for r in items]
        agg_rows.append({
            "algorithm": algo,
            "config": cfg,
            "runs": len(items),
            "best": max(values),
            "worst": min(values),
            "mean": round(statistics.mean(values), 2),
            "stdev": round(statistics.stdev(values), 2) if len(values) > 1 else
0.0,
            "mean_duration_s": round(statistics.mean(durations), 3),
        })
```

2.1.6 File generator_testcase.py

File testcase_generator bertanggung jawab untuk menggenerate test case berukuran small, medium, dan large dengan randomizer.

```
PRESETS = {
    "small": {
```

```
    "ship_w": 5, "ship_l": 5, "n": 6,
     "size_range": (1, 3), "fee_per_cell": (2, 8), "weight_per_cell": (1,
3),
    "eta_range": (1, 7), "capacity_ratio": 1, "seed": 11,
  },
  "medium": {
    "ship_w": 8, "ship_l": 12, "n": 12,
     "size_range": (1, 5), "fee_per_cell": (2, 8), "weight_per_cell": (1,
3),
    "eta_range": (1, 7), "capacity_ratio": 1, "seed": 22,
  },
  "large": {
    "ship_w": 14, "ship_l": 20, "n": 40,
     "size_range": (1, 6), "fee_per_cell": (2, 8), "weight_per_cell": (1,
3),
    "eta_range": (1, 7), "capacity_ratio": 1, "seed": 33,
  },
}


# Template
```

```
def build_vehicles(rng, n, size_range, fee_per_cell, weight_per_cell,
eta_range):
  vehicles = []
  for i in range(1, n + 1):
    lebar = rng.randint(*size_range)
    panjang = rng.randint(*size_range)
    area = lebar * panjang
    vehicles.append({
```

```
        "id": f"V{i:03d}",
        "width": lebar,
        "length": panjang,
        "fee": max(1, round(area * rng.uniform(*fee_per_cell))),
        "weight": max(1, round(area * rng.uniform(*weight_per_cell))),
        "eta": rng.randint(*eta_range),
      })
  return vehicles

# Fungsi membuat daftar kendaraan baru secara acak beserta spesifikasiny  menggunakan perhitungan parameter rentang ukuran yang telah ditentukan di PRESETS
```

```
def check_quality(problem):
  ship = problem["ship"]
  vehicles = problem["vehicles"]
  issues = []

  total_area = sum(v["width"] * v["length"] for v in vehicles)
  total_weight = sum(v["weight"] for v in vehicles)
  if total_area <= ship["width"] * ship["length"]:
        issues.append("total luas kendaraan tidak melebihi luas geladak")
  if total_weight <= ship["max_capacity"]:
    issues.append("total berat kendaraan tidak melebihi kapasitas")

  for v in vehicles:
        muat_h = v["width"] <= ship["width"] and v["length"] <= ship["length"]
```

```
            muat_v = v["length"] <= ship["width"] and v["width"] <= ship["length"]
        if not (muat_h or muat_v):
            issues.append(f"{v['id']} tidak muat di geladak")
        if v["weight"] > ship["max_capacity"]:
            issues.append(f"{v['id']} lebih berat dari kapasitas kapal")
    return issues
# Fungsiini memastikan total luas dan berat seluruh kendaraan melebihi kapasitas kapal (agar ada kendaraan yang harus diseleksi) dan tidak ada kendaraan tunggal yang ukurannya melebihi batas kapal
```

```
def generate(params, max_tries=200):
    rng = random.Random(params["seed"])
    for _ in range(max_tries):
        vehicles = build_vehicles(
                                rng, params["n"], params["size_range"], params["fee_per_cell"],
            params["weight_per_cell"], params["eta_range"],
        )
        total_weight = sum(v["weight"] for v in vehicles)
        problem = {
            "ship": {
                "width": params["ship_w"],
                "length": params["ship_l"],
                                "max_capacity": max(1, round(total_weight * params["capacity_ratio"])),
            },
            "vehicles": vehicles,
```

```
        }
        if not check_quality(problem):
            return problem
    raise RuntimeError("Tidak berhasil membuat test case yang layak")

# Fungsi embangun test case utuh dengan menggabungkan data kapal dan daftar kendaraan
```

```
def describe(problem):
    ship = problem["ship"]
    vehicles = problem["vehicles"]
    total_luas = sum(v["width"] * v["length"] for v in vehicles)
    total_berat = sum(v["weight"] for v in vehicles)
    luas_geladak = ship["width"] * ship["length"]

                    return (f"{len(vehicles)} kendaraan | luas {total_luas}/{luas_geladak}"
            f" | berat {total_berat}/{ship['max_capacity']}")

# Fungsi informasi jumlah kendaraan, perbandingan total luas kendaraan dengan luas geladak, serta perbandingan total berat dengan kapasitas maksimal
```

```
def validate_with_loader(path):
    sys.path.insert(0, str(ROOT / "src"))
    try:
        from problem import load_problem
    except ImportError:
        print("  (load_problem tidak ditemukan, validasi dilewati)")
        return
```

```
    load_problem(str(path))
    print("  lolos load_problem")
```

```
def main():
    names = sys.argv[1:] or list(PRESETS)
    for name in names:
        if name not in PRESETS:
                        sys.exit(f"Nama  tidak  dikenal:  {name}.  Pilihan:  {',
'.join(PRESETS)}")

    OUT_DIR.mkdir(parents=True, exist_ok=True)
    for name in names:
        problem = generate(PRESETS[name])
        path = OUT_DIR / f"tc_{name}.json"
        with open(path, "w", encoding="utf-8") as f:
            json.dump(problem, f, indent=2)
        print(f"{path}: {describe(problem)}")
        validate_with_loader(path)

# Fungsi yang mastiin gada error
```

2.2 Algoritma *Hill Climbing*

Berikut merupakan fungsi algoritma Hill Climbing:

```
import time

from src.core import objective
from src.neighbors import copy_state, all_neighbors
```

```
from src.problem import random_state


def hill_climbing_sideways(problem, rng, max_sideways=100):
    mulai = random_state(problem, rng)
    sekarang = copy_state(mulai)
    nilai_sekarang = objective(sekarang, problem)

    history = [nilai_sekarang]
    sideways = 0
    iterasi = 0
    evaluasi = 1
    t0 = time.perf_counter()
    total_sideways = 0
    alasan = ""

    while True:
        tetangga = all_neighbors(sekarang, problem)
        if len(tetangga) == 0:
            alasan = "tidak ada tetangga"
            break

        # menghitung nilai tetangga
        nilai = []
        for t in tetangga:
            nilai.append(objective(t, problem))
        evaluasi += len(tetangga)

        # nyari tetangga terbaik, kalau sama pilih acak
```

```
        nilai_terbaik = max(nilai)
        seri = []
        for i in range(len(nilai)):
            if nilai[i] == nilai_terbaik:
                seri.append(i)
        terbaik = tetangga[rng.choice(seri)]

        if nilai_terbaik > nilai_sekarang:
            sekarang = terbaik
            nilai_sekarang = nilai_terbaik
            sideways = 0
                    elif nilai_terbaik == nilai_sekarang and sideways < max_sideways:
            sekarang = terbaik
            sideways += 1
            total_sideways += 1
        else:
            if nilai_terbaik == nilai_sekarang:
                alasan = "jatah sideways habis"
            else:
                alasan = "puncak lokal"
            break

        iterasi += 1
        history.append(nilai_sekarang)

    durasi = time.perf_counter() - t0
        return mulai, sekarang, nilai_sekarang, history, durasi, iterasi, evaluasi, sideways, total_sideways, alasan
```

Fungsi hill_climbing_sideways menerima problem (test case yang dijalankan), rng (untuk membangkitkan state awal dan memilih satu neighbor secara acak saat ada nilai yang sama), dan max_sideways (batas langkah sideways berturut-turut). Batas ini perlu ada karena ruang pencarian persoalan packing banyak mengandung plateau, sehingga tanpa batas algoritma bisa berpindah-pindah di area datar tanpa henti.

Pencarian dimulai dari state awal acak hasil random_state(), yang disalin dengan copy_state() menjadi state kerja agar state awal tetap utuh untuk visualisasi. Di setiap iterasi, all_neighbors() membangkitkan semua neighbor valid, objective-nya dihitung, lalu neighbor tertinggi dipilih (acak jika seri). Jika neighbor terbaik lebih baik dari state sekarang, pencarian berpindah dan counter sideways di-reset. Jika nilainya sama dan jatah sideways masih ada, pencarian tetap berpindah dan counter bertambah. Selain itu, pencarian berhenti.

Nilai objective tiap langkah dicatat ke history untuk plotting, bersama jumlah iterasi, jumlah evaluasi, dan durasi. Pencarian berhenti saat mencapai puncak lokal, jatah sideways habis, atau tidak ada neighbor sama sekali. Fungsi lalu mengembalikan state awal dan akhir, nilai akhir, history, durasi, statistik iterasi dan sideways, serta alasan berhenti.

2.2.1 Hasil Percobaan Hill-Climbing with Sideways Move

Config:

- lgoritma: Hill-Climbing with Sideways Move

- max_sideways: 0, 100, 200
- run 1 → seed 1 | run 2 → seed 2 | run 3 → seed 3
- test case: tc_small.json, tc_medium.json, tc_large.json

Parameter yang divariasikan adalah max_sideways, yaitu batas langkah sideways berturut-turut (pindah ke tetangga bernilai sama) sebelum pencarian dihentikan. Nilai 0 sama dengan hill-climbing biasa tanpa sideways. Untuk setiap kombinasi test case dan max_sideways, algoritma dijalankan 3 kali dengan seed 1, 2, dan 3, sehingga state awal tiap run sama untuk semua nilai max_sideways. Yang dicatat: nilai awal, nilai akhir, durasi, jumlah iterasi sampai berhenti (termasuk langkah sideways), jumlah evaluasi tetangga, total langkah sideways, dan alasan berhenti.

2.2.2 Hasil tc_small.json

Tabel 2.2.2.1 max_sideways = 0

	Run 1	Run 2	Run 3
nilai awal	90	90	133
final_value	90	90	133
durasi (detik)	0,001	0,001	0,001
iterasi	0	0	0
evaluasi	31	30	18
total_sideways	0	0	0
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.2.2 max_sideways = 100

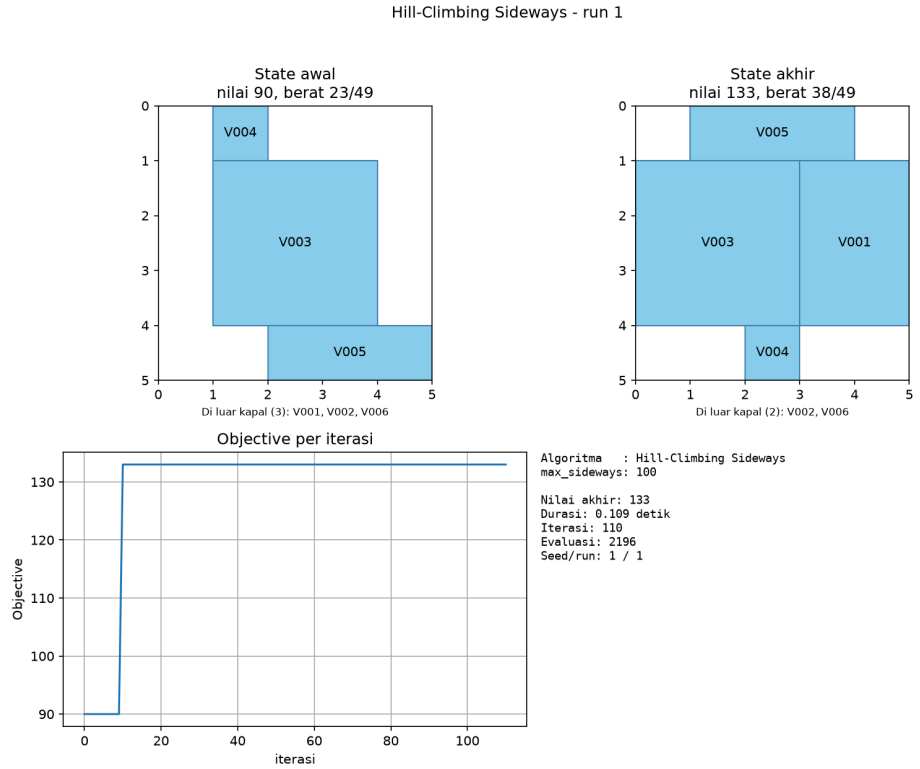
	Run 1	Run 2	Run 3
nilai awal	90	90	133
final_value	90	90	133
durasi (detik)	0,109	0,057	0,053
iterasi	110	105	100
evaluasi	2.196	1.991	1.862
total_sideways	109	104	100
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.2.3 max_sideways = 200

	Run 1	Run 2	Run 3
nilai awal	90	90	133
final_value	133	133	133
durasi (detik)	0,108	0,175	0,107
iterasi	210	205	200
evaluasi	3.976	3.822	3.673
total_sideways	209	204	200
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.2.4 Rekap tc_small

max_sideways	Rata-rata	Min	Maks	Stdev	Durasi rata-rata
0	104,33	90	133	24,83	0,001
100	133,00	133	133	0,00	0,073
200	133,00	133	133	0,00	0,130



2.2.3 Hasil Hasil tc_medium.json

Tabel 2.2.3.1 max_sideways = 0

	Run 1	Run 2	Run 3
nilai awal	192	235	243
final_value	283	278	283

durasi (detik)	0,037	0,028	0,088
iterasi	2	2	2
evaluasi	511	320	340
total_sideways	0	0	0
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.3.2 max_sideways = 100

	Run 1	Run 2	Run 3
nilai awal	192	235	243
final_value	283	278	283
durasi (detik)	1,304	0,985	1,292
iterasi	102	102	102
evaluasi	9.785	10.835	9.320
total_sideways	100	100	100
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

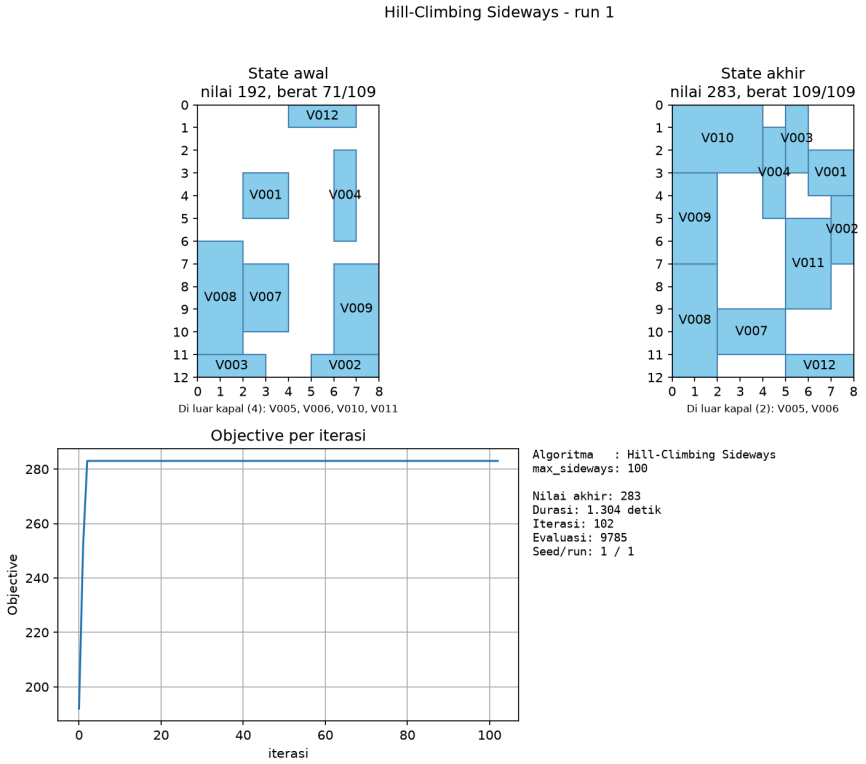
Tabel 2.2.3.3 max_sideways = 200

	Run 1	Run 2	Run 3
nilai awal	192	235	243

final_value	283	278	283
durasi (detik)	2,477	1,907	2,481
iterasi	202	202	202
evaluasi	20.247	21.529	18.850
total_sideways	200	200	200
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.3.4 Rekap tc_small

max_sideways	Rata-rata	Min	Maks	Stdev	Durasi rata-rata
192	235	243	192	235	243
283	278	283	283	278	283
2,477	1,907	2,481	2,477	1,907	2,481



2.2.4 Hasil Hasil tc_large.json

Tabel 2.2.4.1 max_sideways = 0

	Run 1	Run 2	Run 3
nilai awal	910	873	1.002
final_value	1.252	1.160	1.250
durasi (detik)	5,501	6,603	5,108
iterasi	10	10	9
evaluasi	6.347	6.802	6.115
total_sideways	0	0	0
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.4.2 max_sideways = 100

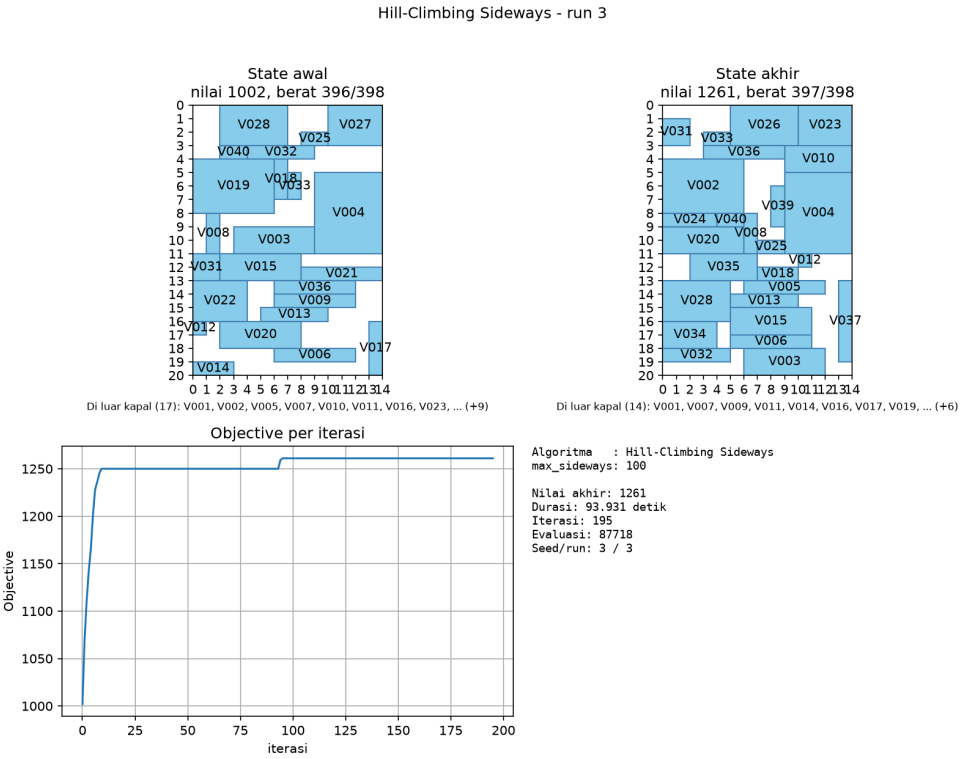
	Run 1	Run 2	Run 3
nilai awal	910	873	1.002
final_value	1.257	1.185	1.261
durasi (detik)	87,675	94,203	93,931
iterasi	194	167	195
evaluasi	103.169	71.711	87.718
total_sideways	183	151	184
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.4.3 max_sideways = 200

	Run 1	Run 2	Run 3
nilai awal	910	873	1.002
final_value	1.257	1.185	1.261
durasi (detik)	87,675	94,203	93,931
iterasi	194	167	195
evaluasi	103.169	71.711	87.718
total_sideways	183	151	184
alasan berhenti	jatah sideways habis	jatah sideways habis	jatah sideways habis

Tabel 2.2.4.4 Rekap tc_small

max_sideways	Rata-rata	Min	Maks	Stdev	Durasi rata-rata
910	873	1.002	910	873	1.002
1.257	1.185	1.261	1.257	1.185	1.261
87,675	94,203	93,931	87,675	94,203	93,931



2.2.5 Analisis

Pengaruh max_sideways terhadap nilai dan durasi paling terlihat pada tc_small. Tanpa sideways (max_sideways = 0), dua dari tiga run terjebak di nilai awal 90 sehingga rata-rata nilai akhirnya hanya 104,33. Dengan 100 maupun 200 langkah sideways, ketiga run mencapai nilai 133. Pada tc_medium, nilai akhir tidak berubah untuk ketiga nilai max_sideways (rata-rata 281,33), tetapi durasi naik dari 0,051 detik menjadi 1,193 detik untuk 100 (sekitar 23 kali lipat) dan

2,288 detik untuk 200 (sekitar 45 kali lipat). Pada tc_large, sideways menaikkan rata-rata nilai dari 1.220,67 menjadi 1.234,33 (naik 13,66 atau sekitar 1,1%), dengan biaya durasi yang naik dari 5,7 detik menjadi 91,9 detik (sekitar 16 kali lipat). Menaikkan max_sideways dari 100 ke 200 tidak mengubah nilai akhir pada test case mana pun, tetapi menambah durasi (pada tc_large naik sekitar 55%). Artinya, nilai maksimum yang dapat dicapai sudah didapat dalam 100 langkah sideways, dan 100 langkah tambahan berikutnya tidak memberi perbaikan.

Dari sisi konsistensi antar seed, pada tc_small dengan max_sideways ≥ 100 simpangan baku (stdev) bernilai 0 karena semua seed menghasilkan nilai yang sama, sedangkan tanpa sideways stdev mencapai 24,83 karena hasil sangat bergantung pada state awal. Pada tc_medium dan tc_large, run 2 (seed 2) selalu menghasilkan nilai akhir terendah (278 pada tc_medium; 1.160 dan 1.185 pada tc_large), sementara seed 1 dan 3 memberi hasil yang hampir sama. Pada tc_large, state awal seed 2 memang yang terendah (873). Namun pada tc_medium state awal seed 2 bukan yang terendah, sehingga hasil akhir tidak hanya ditentukan oleh nilai awal, tetapi juga oleh bentuk lembah tempat pencarian dimulai. Untuk setiap seed, hasil dengan 100 dan 200 sideways selalu sama persis, dan stdev pada tc_large turun dari 52,55 menjadi 42,77 setelah sideways digunakan.

Seluruh 27 run berhenti dengan alasan “jatah sideways habis”, termasuk ketika max_sideways = 0. Tidak ada satu pun run yang berhenti karena semua tetangganya lebih buruk, sehingga tidak ada yang berakhir di puncak lokal murni. Penyebabnya, hampir selalu ada

tetangga dengan nilai yang sama, misalnya menggeser satu kendaraan ke sel kosong tanpa mengubah total fee, sehingga ruang pencarian berbentuk dataran (plateau). Oleh karena itu, pada max_sideways = 0 hasilnya lebih tepat disebut berhenti di plateau. Perlu dicatat bahwa kolom total_sideways dihitung kumulatif sepanjang pencarian, sedangkan max_sideways hanya membatasi langkah sideways berturut-turut tanpa perbaikan. Karena penghitung berturut-turut diatur ulang setiap kali ada perbaikan, total_sideways pada tc_large (183, 151, dan 184) dapat melebihi 100.

2.3 Algoritma *Simulated Annealing*

Berikut merupakan fungsi algoritma simulated annealing:

```
import math
import time
from src.core import objective
from src.neighbors import random_neighbor

def count_stuck(history, treshold):
    countt = 0
    stuckk = 0
    length = 1

    for k in range (1, len(history)):
        if history[k] == history[k - 1]:
            length +=1

        else:
```

```
            if length >= treshold:
                countt += 1
                stuckk += length
            length = 1

    if length >= treshold:
        countt += 1
        stuckk += length

    return countt, stuckk

# Fungsi bantuan untuk menghitung frekuensi 'stuck' di local optima dengan menghitung berapa kali history_fee berurutan memiliki nilai sama. Countt merupakan berapa kali terjadi berutuan, stuckk merupakan berapa banyak iterasi yang terlibat.

def SA(state, problem, temperature, alpha, iterations, rng):
    curr_state = state
    curr_fee = objective(curr_state, problem)
    best_state = curr_state
    best_fee = curr_fee
    history_fee = []
    history_probability = []

    start = time.perf_counter()
    for x in range(iterations):
        neighbor = random_neighbor(curr_state, problem, rng)
        if neighbor is None:
            break
```

```
    neighbor_fee = objective(neighbor, problem)

    delta = neighbor_fee - curr_fee
    if delta >= 0:
      probability = 1
      curr_state = neighbor
      curr_fee = neighbor_fee
      if neighbor_fee > best_fee:
        best_state = neighbor
        best_fee = neighbor_fee

    else:
      probability = math.exp(delta/temperature)
      if rng.random() < probability:
        curr_state = neighbor
        curr_fee = neighbor_fee

    history_probability.append(probability)
    history_fee.append(curr_fee)
    temperature *= alpha

    if temperature <= 0.05:
      break
  end = time.perf_counter()
  durasi = end - start
      countt, stuckk = count_stuck(history_fee, max(5, 0.05 *
len(history_fee)))
        return state, curr_state, curr_fee, history_fee, durasi,
history_probability, countt, stuckk
```

Fungsi diawali dengan menerima parameter berupa state, problem, temperature, alpha, iterations, dan rng

State dan problem merupakan test case yang digunakan saat menjalankan algoritma simulated annealing ini. Temperature merupakan variabel yang digunakan algoritma SA untuk menentukan probabilitas penerimaan neighbor yang memiliki hasil objective function yang lebih rendah. Alpha merupakan *cooling_rate* yang digunakan untuk mengurangi besar alpha pada setiap iterasi karena karakteristik dari SA sendiri akan menjadi semakin greedy selama proses iterasi (semakin menolak neighbor buruk). Iterations merupakan maksimal iterasi digunakan untuk menjadi stop condition tambahan.

Di awal dimulai dengan menghitung *curr_state* dan *curr_fee* dari state yang diterima sebagai parameter. Kemudian masuk ke dalam iterasi. Dalam iterasi akan dipilih neighbor acak dengan fungsi random_neighbor(). Lalu, dihitung nilai objective function dari neighbor. Jika lebih baik atau sama dengan maka neighbor akan selalu diterima. Jika lebih buruk, maka akan terjadi perhitungan dengan rumus e^(delta/temperature) untuk menghitung probabilitas penerimaan.

Setelah itu nilai objective function dan nilai e^(delta/temperature) dicatat untuk keperluan output dan plotting. Kemudian, nilai temperature dikalikan dengan *cooling_rate*. Temperature < 0.05 dipilih sebagai stop condition karena nilai tersebut sudah menunjukkan temperature sudah bernilai sangat kecil.

Jika *stop condition* sudah terpenuhi maka iterasi akan berakhir dan mengembalikan *output* yang diperlukan. Countt merupakan berapa kali iterasi 'stuck' atau membentuk garis lurus. Stuckk dikemablikan hanya untuk berjaga-jaga.

2.3.1 Hasil Percobaan Simulated Annealing

Untuk percobaan Simulated Annealing, config yang dipilih adalah sebagai berikut:

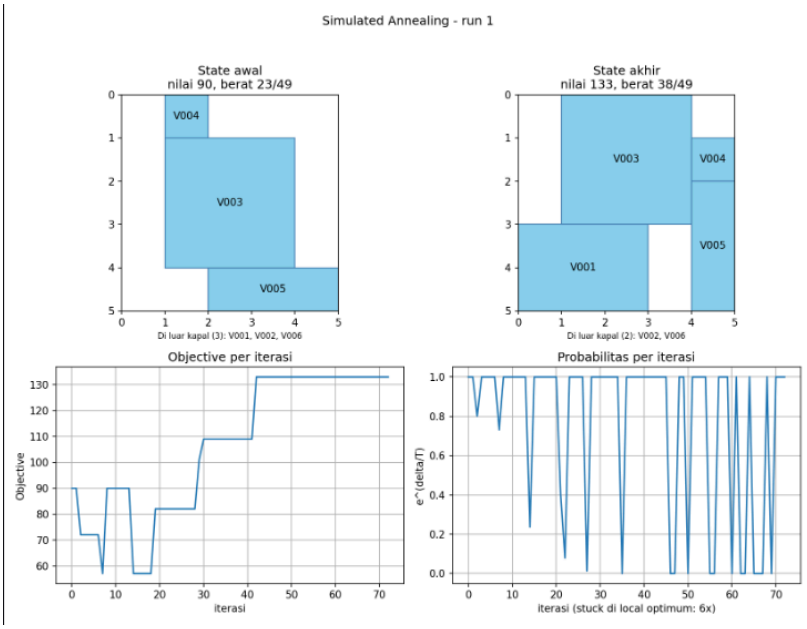
Temperature: 100
Alpha/Cooling_rate = 0.9 / 0.925 / 0.95
Iterations = 5000
Dengan masing-masing iterasi dijalankan 3 kali dengan seed yang berbeda ([1, 2, 3)]untuk mengetahui konsistensi.

2.3.1.1 Percobaan Terhadap Perbedaan State Awal

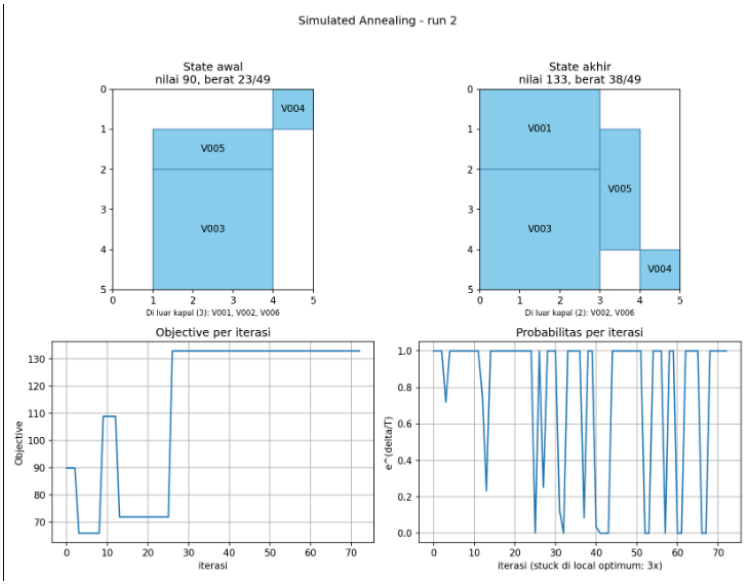
Untuk percobaan ini diukur dengan ukuran state awal yang berbeda, dengan ketentuan state di Bab I. Demi memudahkan perbandingan, maka nilai cooling_rate pada percobaan yang dilampirkan pada laporan ini akan ditetapkan pada 0.9.

Tabel 2.3.1.1.1 Hasil tc_small.json

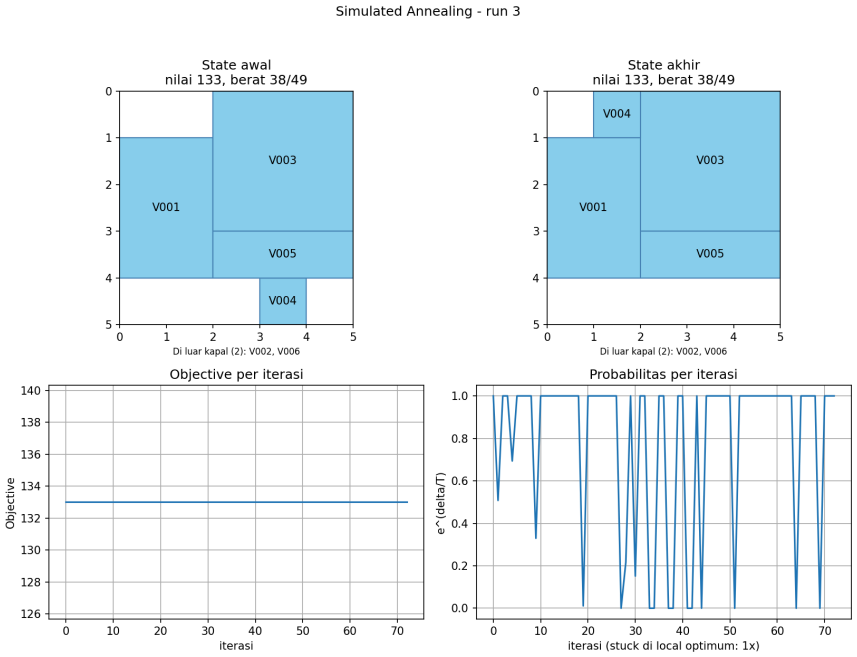
	Run 1	Run 2	Run 3
final_value	133	133	133
duration_s	0.125	0.122	0.171
iteration	73	73	73
count_stuck	6	3	1



Gambar 2.3.1.1.2 Hasil Run 1 tc_small.json



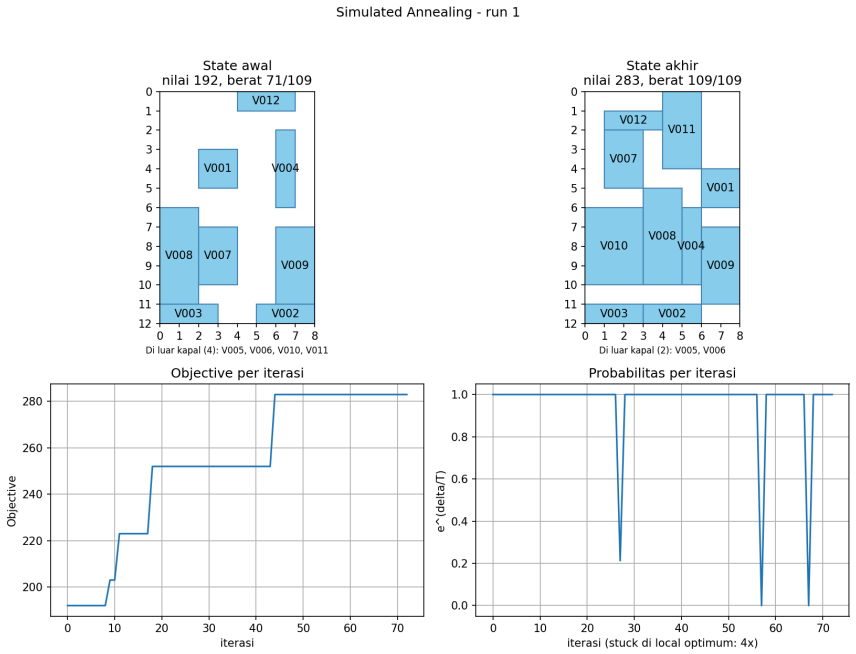
Gambar 2.3.1.1.3 Hasil Run 2 tc_small.json



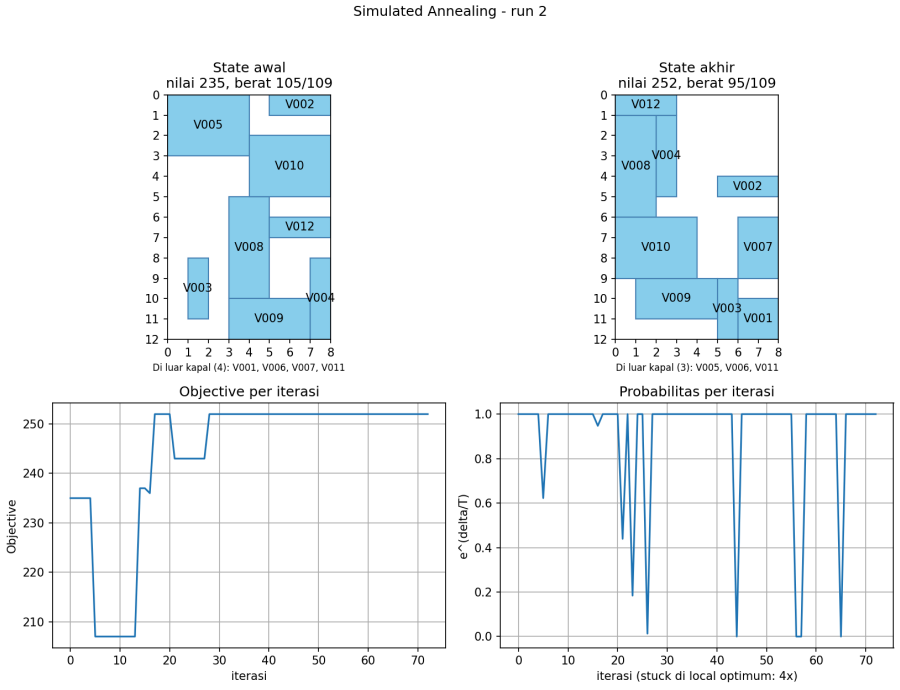
Gambar 2.3.1.1.4 Hasil Run 3 tc_small.json

Tabel 2.3.1.1.5 Hasil tc_medium.json

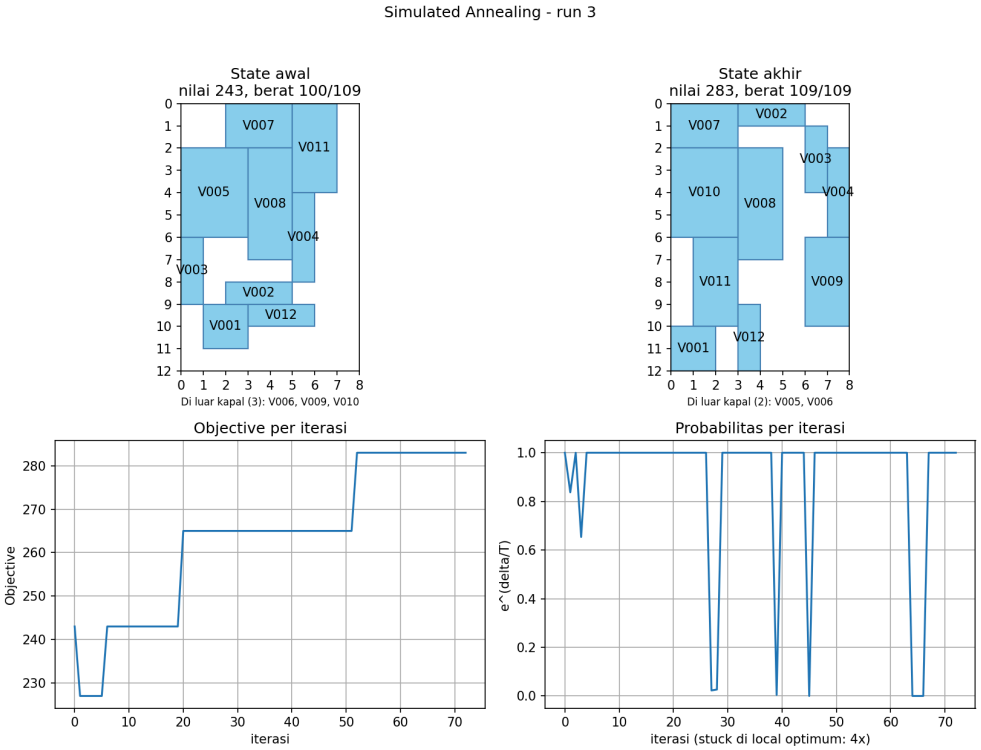
	Run 1	Run 2	Run 3
final_value	283	252	283
duration_s	2.499	2.348	2.236
iteration	73	73	73
count_stuck	4	2	4



Gambar 2.3.1.1.6 Hasil Run 1 tc_medium.json



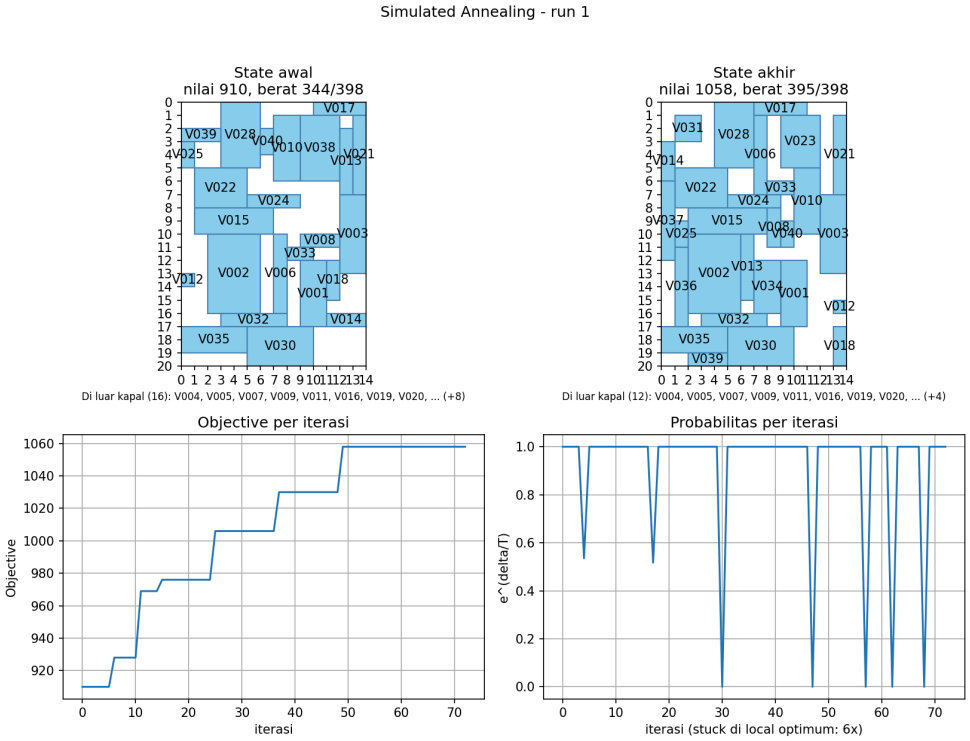
Gambar 2.3.1.1.7 Hasil Run 2 tc_medium.json



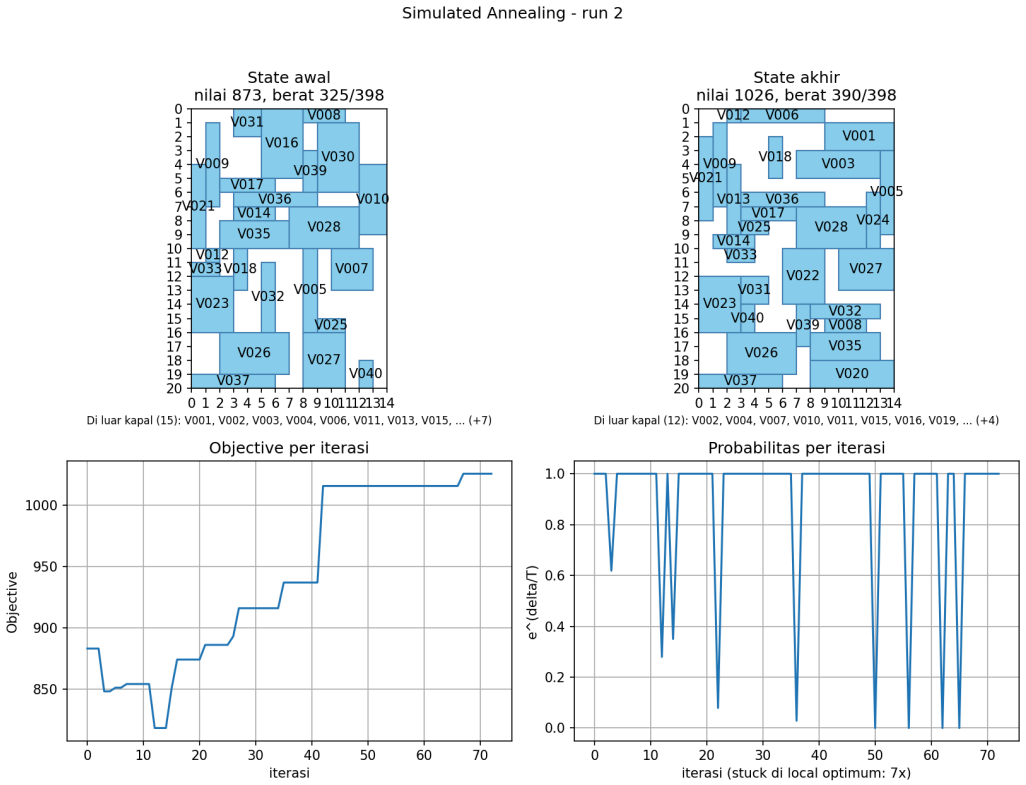
Gambar 2.3.1.1.8 Hasil Run 3 tc_medium.json

Tabel 2.3.1.1.9 Hasil tc_large.json

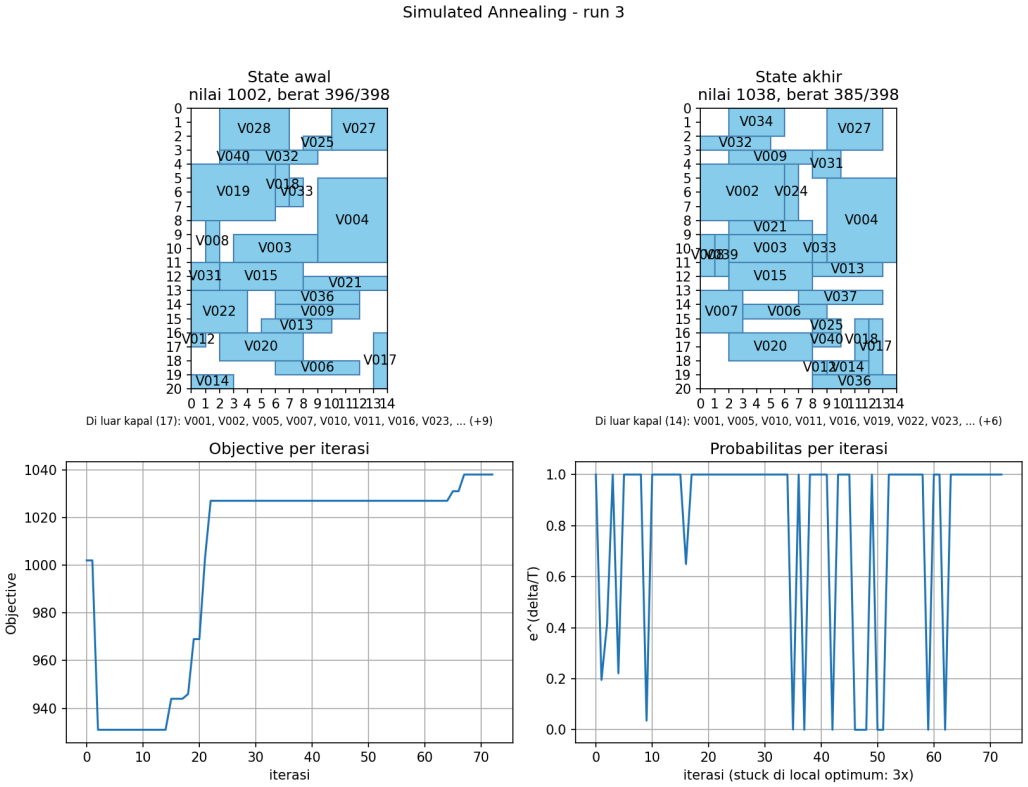
	Run 1	Run 2	Run 3
final_value	1058	1026	1038
duration_s	184.471	179.261	94.712
iteration	73	73	73
count_stuck	6	7	3



Gambar 2.3.1.1.10 Hasil Run 1 tc_large.json



Gambar 2.3.1.1.11 Hasil Run 2 tc_large.json



Gambar 2.3.1.1.12 Hasil Run 3 tc_large.json

Analisis

Berdasarkan eksperimen dengan mengubah ukuran test case didapatkan bahwa durasi dari algoritma simulated annealing dapat meningkat sangat tajam. Hal ini karena semakin besar test case maka semakin besar state yang valid sehingga cost dari setiap iterasi menjadi lebih besar. Hal ini dapat dioptimasi dengan algoritma random_neighbor() yang lebih baik. Selain itu, ukuran test case yang lebih kecil menunjukkan konsistensi hasil yang lebih baik.

2.3.1.2 Percobaan Terhadap Perbedaan Cooling Rate

Untuk percobaan ini diukur dengan nilai cooling_rate atau alpha yang berbeda [0.90, 0.925, 0.95]. Demi memudahkan perbandingan, maka perbandingan akan dilakukan di ukuran test_case yang sama, yaitu tc_medium.json.

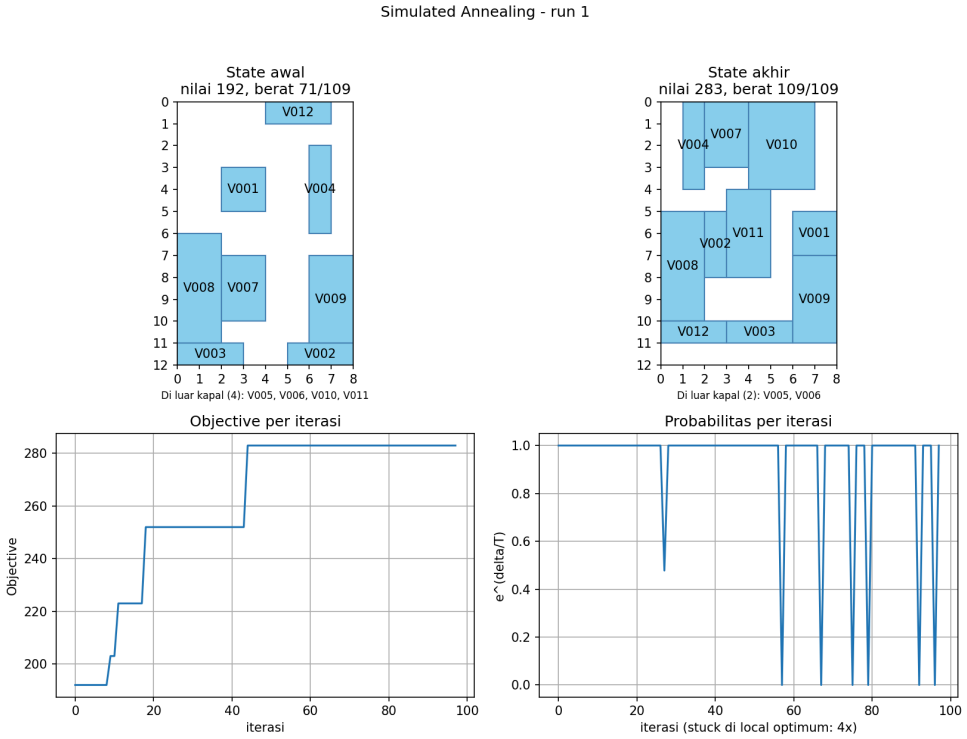
Tabel 2.3.1.2.1 Hasil c = 0.90

	Run 1	Run 2	Run 3
final_value	283	252	283
duration_s	2.499	2.348	2.236
iteration	73	73	73
count_stuck	4	2	4

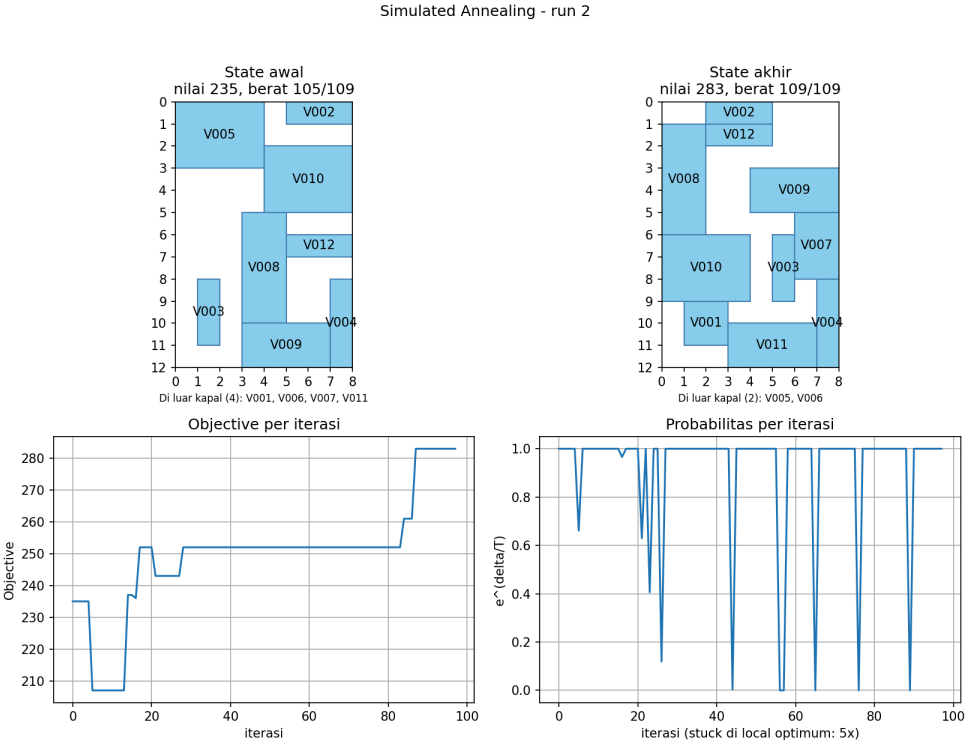
[Note: gambar sama dengan 2.3.1.1.6 hingga 2.3.1.1.7

Tabel 2.3.1.2.2 Hasil c = 0.925

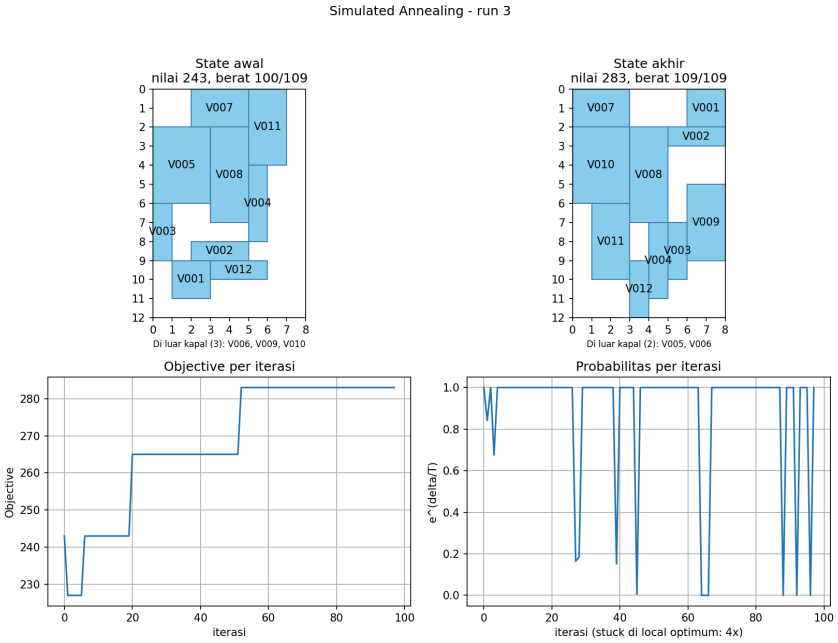
	Run 1	Run 2	Run 3
final_value	283	283	283
duration_s	3.440	3.432	3.198
iteration	98	98	98
count_stuck	4	5	4



Gambar 2.3.1.2.3 Hasil Run 1 c = 0.925



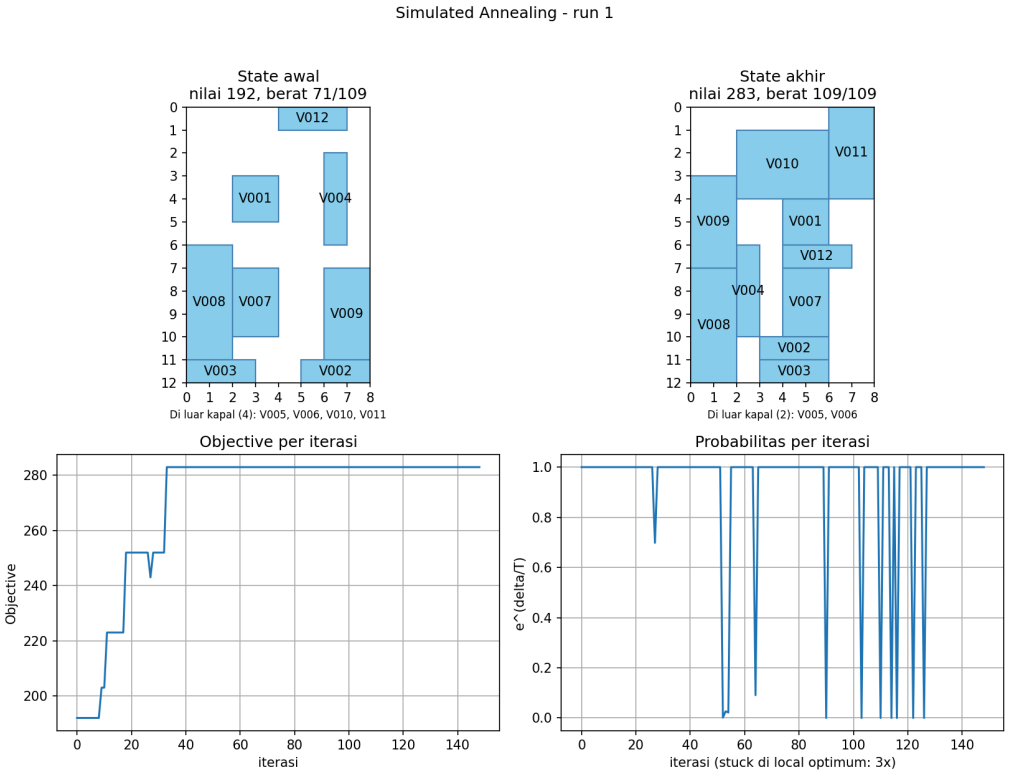
Gambar 2.3.1.2.4 Hasil Run 2 c = 0.925



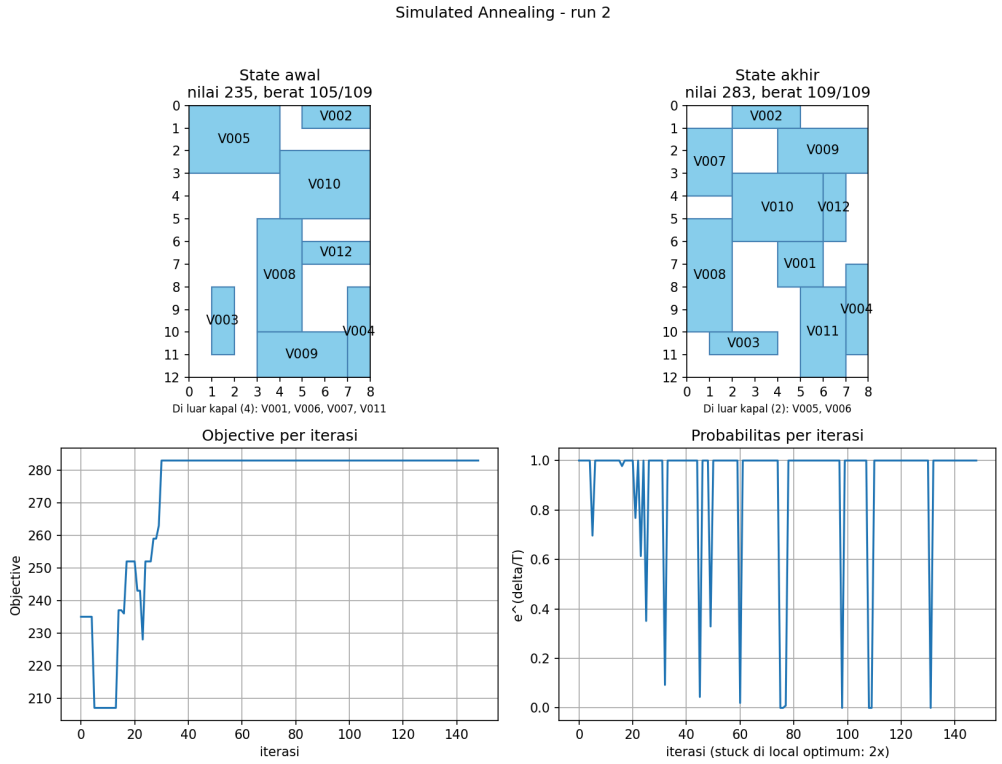
Gambar 2.3.1.2.4 Hasil Run 3 c = 0.925

Tabel 2.3.1.2.5 Hasil c = 0.95

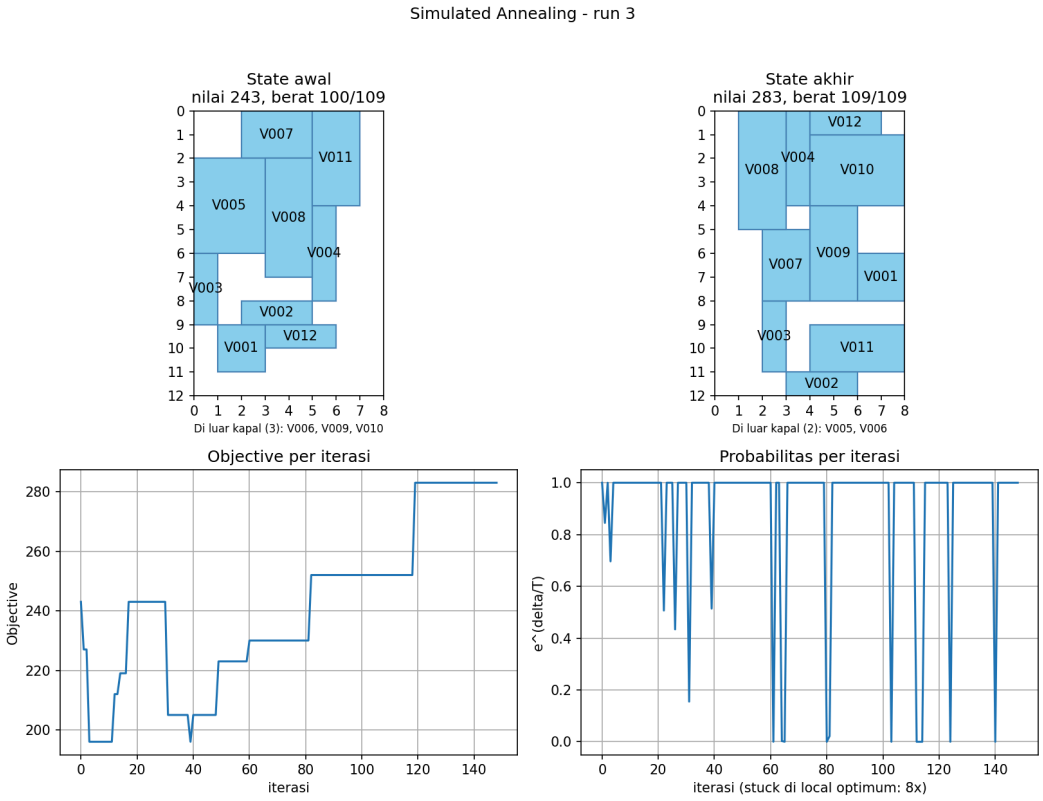
	Run 1	Run 2	Run 3
final_value	283	283	283
duration_s	5.360	4.992	5.269
iteration	149	149	149
count_stuck	3	2	8



Gambar 2.3.1.2.6 Hasil Run 1 c = 0.95



Gambar 2.3.1.2.7 Hasil Run 2 c = 0.95



Gambar 2.3.1.2.8 Hasil Run 3 c = 0.95

Analisis

Berdasarkan eksperimen dengan mengubah nilai cooling rate, semakin kecil cooling rate, maka jumlah iterasi akan menjadi lebih kecil. Begitu juga sebaliknya dengan semakin besarnya cooling rate akan memperlama durasi akibat iterasi yang lebih banyak (iterasi yang lebih banyak disebabkan oleh pengurangan nilai T berlangsung dengan lebih lambat). Namun, di sisi lain dengan bertambahnya iterasi, memberikan lebih banyak kesempatan untuk keluar dari local optima sehingga hasil dapat lebih konsisten dibuktikan dengan c = 0.9 yang memiliki 1 nilai yang tidak konsisten pada run 2 yaitu 252.

2.3.1.3 Percobaan Keseluruhan

Bagian ini digunakan untuk menunjukkan ringkasan untuk seluruh 27 eksperimen (3 seed * 3 ukuran * 3 cooling rate)

Tabel 2.3.1.3.1 Ringkasan tc_small.json

cooling_rate	Rata-rata	Min	Maks	Stdev	Durasi rata-rata
0.90	133	133	133	0	0.140
0.925	133	133	133	0	0.141
0.95	133	133	133	0	0.212

Tabel 2.3.1.3.2 Ringkasan tc_medium.json

cooling_rate	Rata-rata	Min	Maks	Stdev	Durasi rata-rata
0.90	272.67	252	283	17.9	2.361
0.925	283	283	283	0	3.357
0.95	283	283	283	0	5.207

Tabel 2.3.1.3.3 Ringkasan tc_large.json

cooling_rate	Rata-rata	Min	Maks	Stdev	Durasi rata-rata
0.90	1040.67	1026	1058	16.17	152.815
0.925	1041.67	1006	1061	30.92	225.667
0.95	1070.67	1048	1094	23.01	352.019

2.3.1.4 Analisis Keseluruhan

Pada tc_small, semua run menghasilkan 133 sehingga test case ini masih belum dapat membedakan pengaruh antar konfigurasi. Pada tc_medium, c = 0.925 dan 0.95 selalu menghasilkan 283, sedangkan 0.9 menghasilkan satu run lebih rendah yaitu 252. Pada tc_large, rata-rata untuk 0.90, 0.925, dan 0.95 adalah 1040.67, 1041.67, dan 1070.67 dengan nilai terbaik pada 1095 pada c = 0.95.

Durasi meningkat sangat tajam seiring bertambahnya ukuran meskipun jumlah iterasinya sama. Pada c = 0.9 rata-ratanya 0.14 s, 2.36 s, dan 152.8 s (small, medium, large). Hal ini mungkin disebabkan oleh algoritma fungsi random_neighbor() yang belum dioptimasi dan masih memanggil all_neighbor() sehingga ketika test case menjadi besar dan jumlah state yang valid sangat banyak, maka cost pada setiap iterasi akan semakin besar. Pengaruh cooling_rate bergantung pada ukuran test case. Semakin mudah maka pengaruh semakin tidak terlihat dan sebaliknya semakin besar maka pengaruhnya akan semakin terlihat.

Algoritma SA yang memungkinkan memilih neighbor dengan value yang lebih buruk memberikan kesempatan lebih untuk keluar dari local optimum.

2.4 Algoritma *Genetic*

Berikut adalah fungsi-fungsi yang digunakan dalam algoritma genetic.

Fungsi inisialisasi
def create_population(problem, rng, population_size):

```
    list_of_states = []
    for _ in range(population_size):
        state = random_state(problem, rng)
        list_of_states.append(state)
        list_of_fitness = [objective(state, problem) for state in list_of_states]
    return list_of_states, list_of_fitness

# Fungsi untuk inisialisasi populasi awal. Fungsi ini akan memanggil random_state beberapa kali untuk mencetak susunan kapal random sebanyak populasi yang diminta. Tiap susunan akan dilihat fitnessnya menggunakan objective.
```

Fungsi selection

```
def tournament(list_of_fitness, list_of_states,rng, prohibited = None, tournament_size = 3):
    population_size = len(list_of_states)
    valid_index = [i for i in range(population_size) if prohibited is None or i not in prohibited]
    tournament_index = rng.sample(valid_index, min(tournament_size, len(valid_index)))
    best_index = tournament_index[0]
    best_fitness = list_of_fitness[best_index]
    for i in tournament_index:
        if list_of_fitness[i] > best_fitness:
            best_index = i
            best_fitness = list_of_fitness[i]
    return best_index, list_of_states[best_index], best_fitness
```

Fungsi untuk memilih calon parent. Dipilih dengan teknik selection tournament. Teknik selection ini memilih kandidat parent dengan menciptakan simulasi beberapa kompetisi di antara state yang terpilih. Pemenang yang dipilih adalah state dengan nilai fitness lebih besar. Kompetisi yang dilakukan pada sistem kami adalah sebanyak 3 kali (diperoleh 3 parent yang sama)
Fungsi crossover
def crossover(parent1, parent2, rng):   crossover_point = rng.randint(1, len(parent1) - 1)   child = copy.deepcopy(parent1[0:crossover_point]) + copy.deepcopy(parent2[crossover_point:])   return child # Fungsi yang memotong parent1 dan parent2 di titik random tertentu. Hasil potongan awal parent1 digabungkan dengan potongan akhir parent2 untuk menciptakan anak.
Fungsi mutation

def mutation(child, problem, rng, mutation_rate=0.1):
  # kalau mutation rate lebih kecil di banding random number, mutasi ga terjadi, langsung return child aja
  if rng.random() > mutation_rate:
  return child
 	
  i = rng.randint(0, len(child) - 1)
  origin_state = child[i].copy()
  vehicle_spec = problem["vehicles"][i]
  ship_width = problem["ship"]["width"]

```
ship_length = problem["ship"]["length"]

# kendaraan di luar kapal cuma bisa mutasi lokasi doang
if origin_state["location"] == -1:
    category = "location"
else:
    category = rng.choice(["location", "x", "y", "orientation"])

# mutasi tiap2 kategori dalam state secara random
if category == "location":
    if origin_state["location"] == 0:
        child[i] = {"location": -1, "x": 0, "y": 0, "orientation" : "H"}
    else:
        orientation = rng.choice(["H", "V"])
        if orientation == "H":
            limit_x = ship_width - vehicle_spec["width"]
            limit_y = ship_length - vehicle_spec["length"]
        else:
            limit_x = ship_width - vehicle_spec["length"]
            limit_y = ship_length - vehicle_spec["width"]
        child[i] = {
            "location": 0,
            "x": rng.randint(0, max(0, limit_x)),
            "y": rng.randint(0, max(0, limit_y)),
            "orientation": orientation
        }
elif category == "x":
    if child[i]["orientation"] == "H":
        limit_x = ship_width - vehicle_spec["width"]
```

```
        else:
            limit_x = ship_width - vehicle_spec["length"]
        child[i]["x"] = rng.randint(0, max(0, limit_x))
    elif category == "y":
        if child[i]["orientation"] == "H":
            limit_y = ship_length - vehicle_spec["length"]
        else:
            limit_y = ship_length - vehicle_spec["width"]
        child[i]["y"] = rng.randint(0, max(0, limit_y))
    elif category == "orientation":
        child[i]["orientation"] = "H" if child[i]["orientation"] == "V" else "V"

     # make sure validasi child, bcs child bisa aja jadi ga bisa dimuat di kapal setelah mutasi
    if not is_valid(child, problem):
        child[i] = origin_state

    return child

# Fungsi ini memberikan variasi random pada susunan anak yang baru lahir selama lolos mutation-rate. Satu kendaraan akan dipilih secara random untuk diubah statenya. Kalau kendaraan ada diluar kapal -> mutasi yang dicoba hanya ubah lokasi. Tapi kalau kendaraan ada di dalam kapal -> sistem akan acak salah satu atribut. Kalau hasil randomisasi ternyata membuat kendaraan tidak memenuhi is_valid, maka susunan akan dibatalkan dan dikembalikan seperti semla.
```

Fungsi mating

```
# fungsi mating. fungsi buat ngeprodiksi satu anak yang valid
```

```
def mating (list_of_states, list_of_fitness, problem, rng, mutation_rate, tournament_size, max_attempts=10):
  for _ in range(max_attempts):
            pi1 = tournament(list_of_fitness, list_of_states, rng, tournament_size=tournament_size)[0]
            pi2 = tournament(list_of_fitness, list_of_states, rng, prohibited=[pi1], tournament_size=tournament_size)[0]
    child = crossover(list_of_states[pi1], list_of_states[pi2], rng)
    mutant = mutation(child, problem, rng, mutation_rate)
    if is_valid(mutant, problem):
      return mutant
  return copy.deepcopy(list_of_states[rng.choice([pi1, pi2])])

# Fungsi ini memproduksi tepat satu anak baru yang susunannya divalidasi layak atau tidak. Fungsi menyeleksi 2 parent unggul lewat turnamen dan dikawinkan dengan crossover, lalu diberi variasi acak lewat mutation. Hal ini diulang sebanyak max_attempts sampai anak yang valid ditemukan. Kalau batas percobaan habis dan anak valid tidak tercapai, maka fungsi akan mereturn salah satu parent aslinya.
```

Fungsi utama genetic algorithm

```
#fungsi utama genetic algorithm
def genetic_algorithm(problem, rng, population_size=50, max_generations=100, mutation_rate=0.1, tournament_size=3, max_attempts=10):
  # ibarat taksi nyalain argo yee
  start_time = time.perf_counter()
  evaluations = 0
```

```
    # inisialisasi populasi. bikin generasi ke 0
        list_of_states, list_of_fitness = create_population(problem, rng, population_size)
    evaluations += population_size

    # ngecek siapa si yang paling mantep
      best_state, best_fitness = max(zip(list_of_states, list_of_fitness), key=lambda x: x[1])
    initial_state = copy.deepcopy(best_state)
    history_max = []
    history_mean = []

    # proses looping utama
    for generation in range(max_generations):
        new_list_of_states = []
        new_list_of_fitness = []

        # looping buat bikin anak
        for _ in range(population_size):
                child = mating(list_of_states, list_of_fitness, problem, rng, mutation_rate, tournament_size, max_attempts)
            new_list_of_states.append(child)
            new_list_of_fitness.append(objective(child, problem))
            evaluations += 1

        # list dah penuh, populasi lama diganti sama populasi anak2 baru
        list_of_states = new_list_of_states
        list_of_fitness = new_list_of_fitness
```

```
                current_best_state, current_best_fitness = max(zip(list_of_states, list_of_fitness), key=lambda x: x[1])
        if current_best_fitness > best_fitness:
            best_state = copy.deepcopy(current_best_state)
            best_fitness = current_best_fitness

        history_max.append(current_best_fitness)
        history_mean.append(sum(list_of_fitness) / len(list_of_fitness))

    end_time = time.perf_counter()

    return{
        "algorithm": "Genetic Algorithm",
        "config": {
            "population_size": population_size,
            "max_generations": max_generations,
            "mutation_rate": mutation_rate,
            "tournament_size": tournament_size,
            "max_attempts": max_attempts
        },
        "initial_state": initial_state,
        "final_state": best_state,
        "final_value": best_fitness,
        "history_max": history_max,
        "history_mean": history_mean,
        "duration_s": end_time - start_time,
        "iterations": max_generations,
        "evaluations": evaluations,
        "extra": {}
```

```
}
```

Fungsi ini menjalankan seluruh siklus evolusi. Pertama-tama timer dijalankan untuk menghitung durasi. Lalu fungsi mencetak populasi generasi awal untuk mencari susunan anak sementara. Selanjutnya sistem masuk ke looping utama sebanyak batas generasi yang ditentukan. Di sini, sistem memanggil mating untuk memproduksi anak bru hingga populasi penuh, lalu populasi lama dibuang dan diganti dengan populasi anak. Setelah generasi berganti, algoritma akan mengecek best-state dan mencatat riwayat perkembangan history_max dan history_mean untuk dibuat plot. Saat seluruh siklus selesai, timer mati dan membungkus semua data eksperimen dalam satu DICT.

2.4.1 Hasil Percobaan 1: Jumlah Populasi sebagai Kontrol

Config:

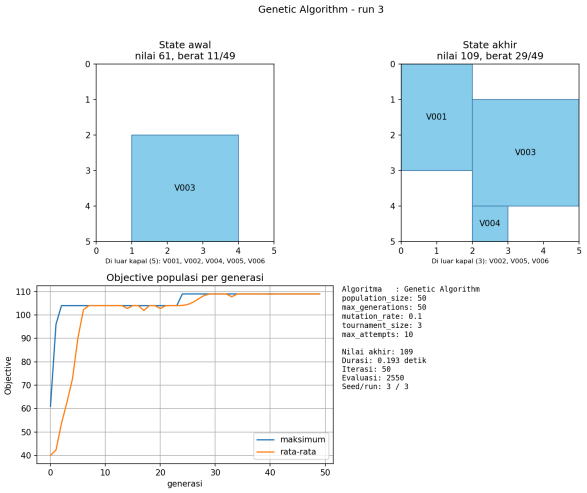
- max_attempts : 10
- max_generations: 50, 100, dan 200 (variasi iterasi)
- mutation_rate: 0.1
- population_size: 50 (jumlah populasi)
- run 1 -> seed 1
- run 2 -> seed 2
- run 3 -> seed 3

Kami menggunakan ukuran input yang berbeda untuk tiap eksperimen (jenis-jenis input dapat dilihat di Bab I)

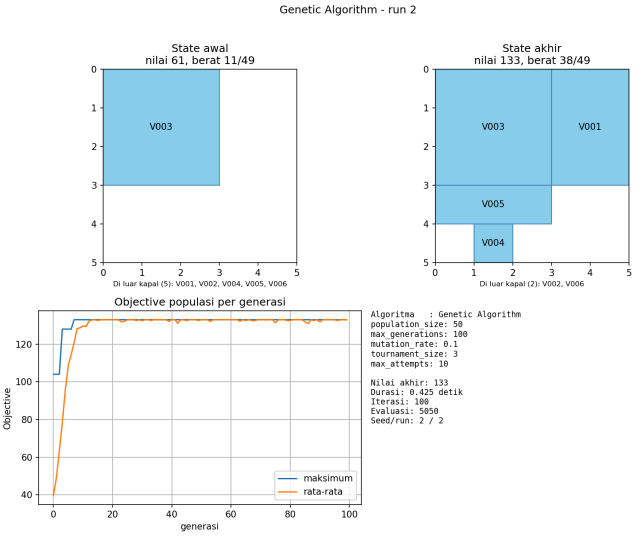
Tabel 2.4.1.1 Hasil Percobaan 1: Jumlah Populasi sebagai Kontrol

Hasil Percobaan 1: Input tc_small.json (hanya dipilih yang objectivenya paling baik, untuk lengkapnya bisa dilihat di sini)			
	Iterasi 50	Iterasi 100	Iterasi 200
State awal	Dapat dilihat di github		
State akhir			
Objective akhir	133	133	133
Jumlah populasi	50	50	50
Durasi	0.193 detik	0,425 detik	0.98 detik
Run/Seed	3/3	2/2	2/2
Final State: Input tc_medium.json (hanya dipilih yang objectivenya paling baik, untuk lengkapnya bisa dilihat di sini)			
	Iterasi 50	Iterasi 100	Iterasi 200
State awal	Dapat dilihat di sini		
State akhir			

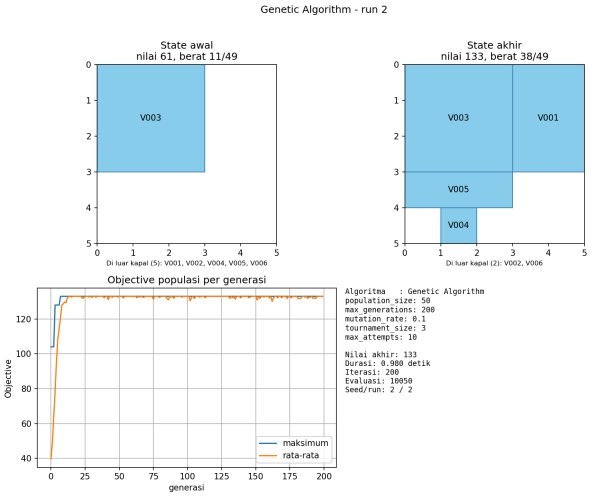
Objective akhir	283	283	283
Jumlah populasi	50	50	50
Durasi	0.51 detik	0.819 detik	1.69 detik
Run/Seed	1/1	1/1	1/1
Final State: Input tc_large.json (hanya dipilih yang objectivenya paling baik, untuk lengkapnya bisa dilihat di sini)			
	Iterasi 50	Iterasi 100	Iterasi 200
State awal	Dapat dilihat di sini		
State akhir			
Objective akhir	1206	1226	1226
Jumlah populasi	50	50	50
Durasi	3.909 detik	6.213	11.542
Run/Seed	1/1	1/1	1/1



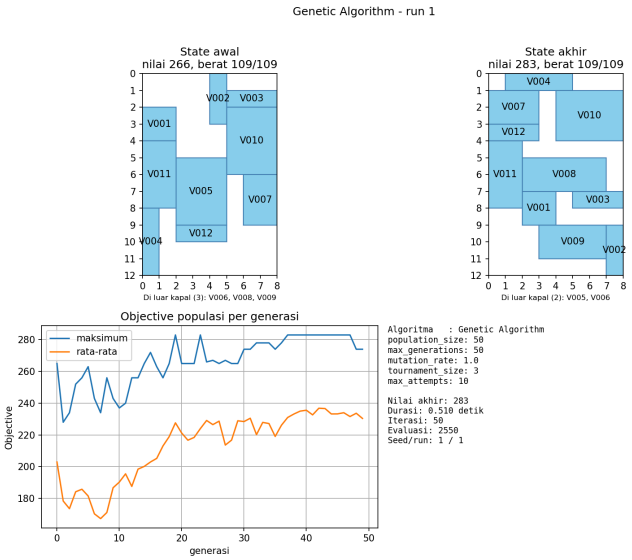
Gambar 2.4.1.1 Percobaan 1: Input tc_small.json iterasi 50 run 3/3



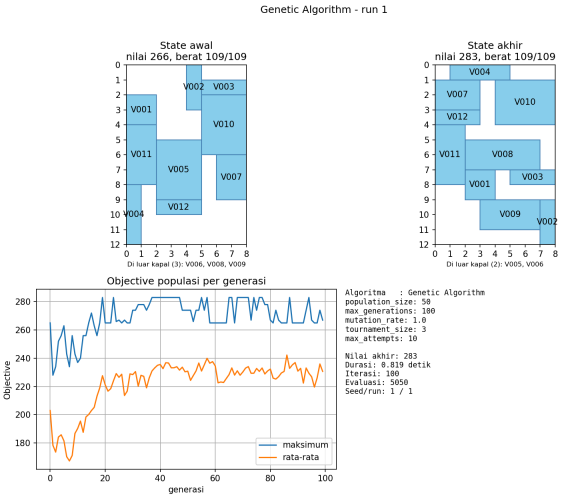
Gambar 2.4.1.2 Percobaan 1: Input tc_small.json iterasi 100 run 2/2



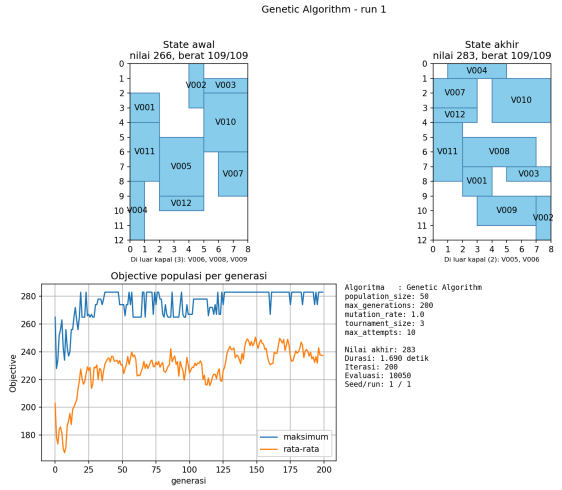
Gambar 2.4.1.3 Percobaan 1: Input tc_small.json iterasi 200 run 2/2



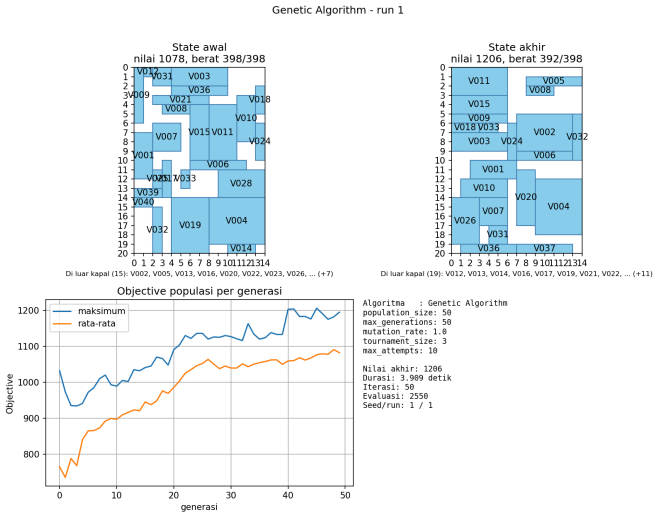
Gambar 2.4.1.4 Percobaan 1: Input tc_medium.json iterasi 50 run 1/1



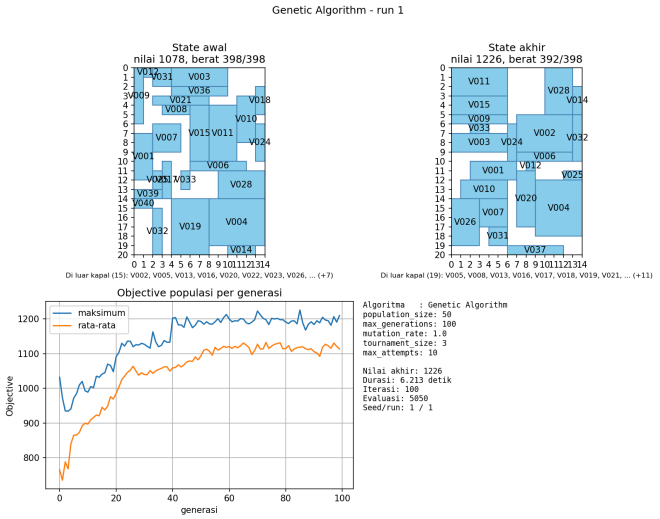
Gambar 2.3.1.5 Percobaan 1: Input tc_medium.json iterasi 100 run 1/1



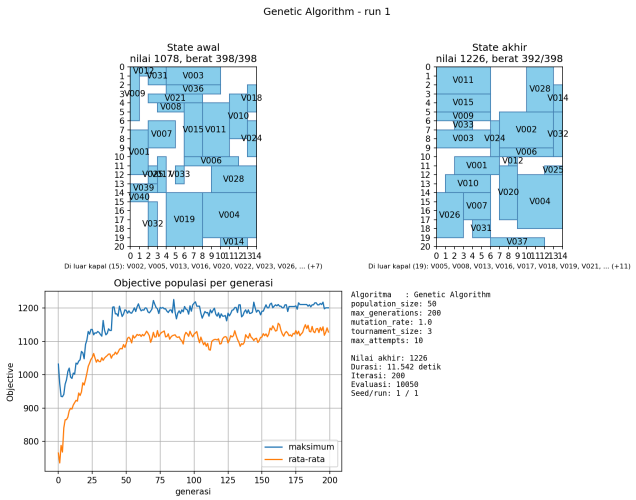
Gambar 2.4.1.6 Percobaan 1: Input tc_medium.json iterasi 200 run 1/1



Gambar 2.4.1.7 Percobaan 1: Input tc_large.json iterasi 50 run 1/1



Gambar 2.4.1.8 Percobaan 1: Input tc_large.json iterasi 100 run 1/1



Gambar 2.4.1.9 Percobaan 1: Input tc_large.json iterasi 200 run 1/1

2.4.2 Hasil Percobaan 2: Jumlah Iterasi sebagai Kontrol

Config:

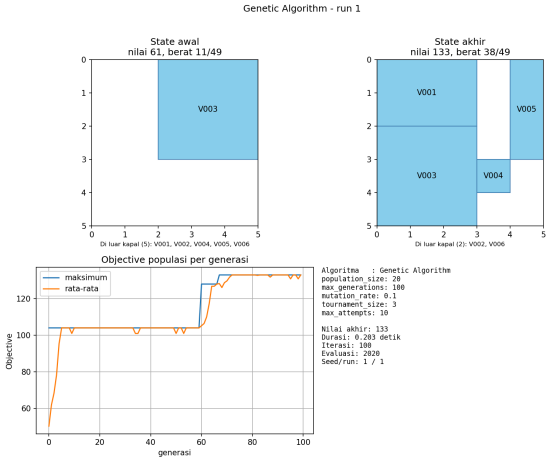
- max_attempts : 10
- max_generations: 100 (jumlah iterasi)
- mutation_rate: 0.1
- Population_size: 20, 50, 100 (variasi populasi)
- run 1 -> seed 1
- run 2 -> seed 2
- run 3 -> seed 3

Tabel 2.4.2.1 Hasil Percoban 2: Jumlah Iterasi Sebagai Kontrol

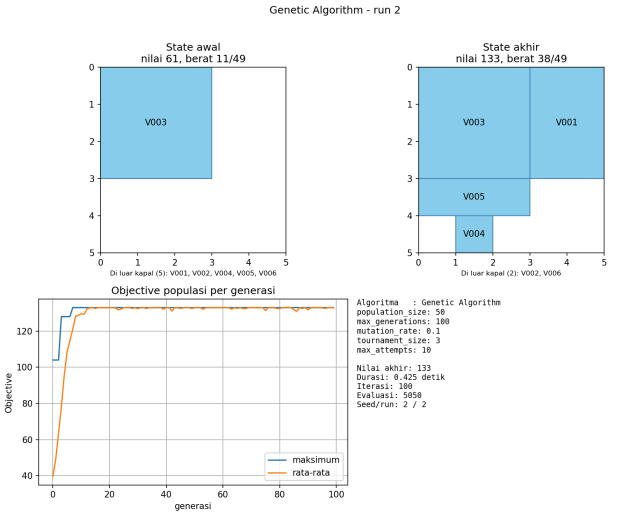
Hasil Percobaan 1: Input tc_small.json (hanya dipilih yang objectivenya paling baik, untuk lengkapnya bisa dilihat di github)			
	Populasi: 20	Populasi: 50	Populasi: 100

State awal	Dapat dilihat di sini		
State akhir			
Objective akhir	133	133	133
Jumlah Iterasi	100	100	100
Durasi	0.203 detik	0.425 detik	1.016 detik
Run/Seed	1/1	2/2	2/2
Final State: Input tc_medium.json (hanya dipilih yang objectivenya paling baik, untuk lengkapnya bisa dilihat di github)			
	Populasi: 20	Populasi: 50	Populasi: 100
State awal	Dapat dilihat di github		
State akhir			
Objective akhir	283	283 (awal = akhir)	283 (awal =akhir)
Jumlah iterasi	100	100	100
Durasi	0.34 detik	0.819 detik	1.832 detik

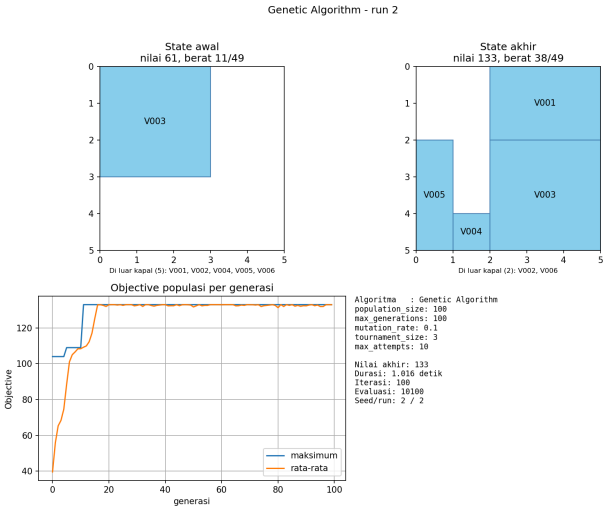
Run/Seed	2/2	1/1	3/3
Final State: Input tc_large.json (hanya dipilih yang objectivenya paling baik, untuk lengkapnya bisa dilihat github)			
	Populasi: 20	Populasi: 50	Populasi: 100
State awal	Dapat dilihat di guthub		
State akhir			
Objective akhir	1174	1226	1247
Jumlah iterasi	100	100	100
Durasi	3.039 detik	6.213 detik	14.834
Run/Seed	2/2	1/1	2/2



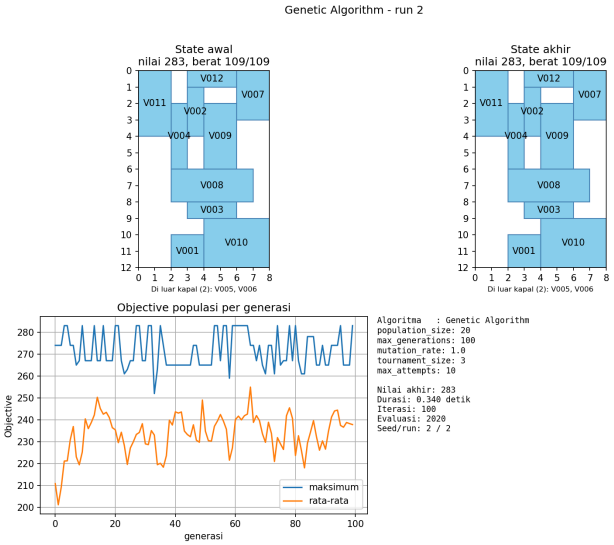
Gambar 2.4.1.1 Percobaan 2: Input tc_small.json populasi: 20 run 1/1



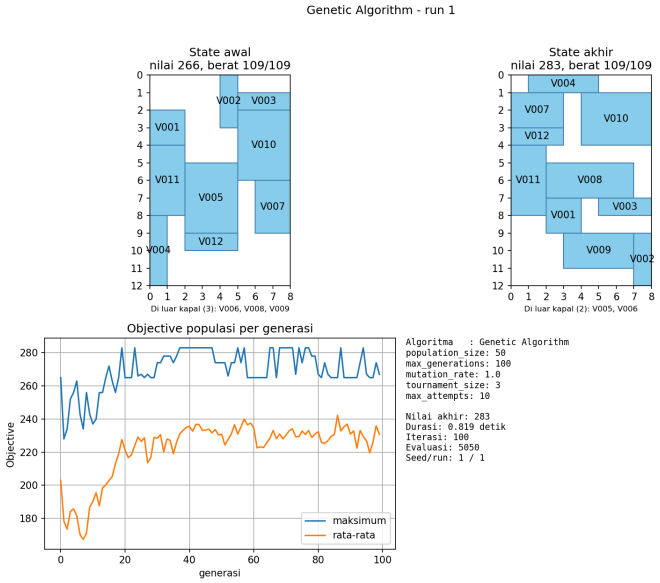
Gambar 2.4.1.2 Percobaan 2: Input tc_small.json populasi: 50 run 2/2



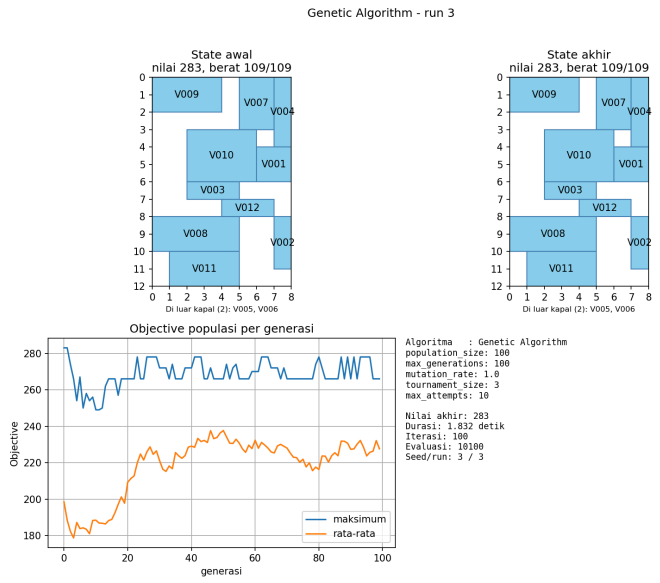
Gambar 2.4.1.3 Percobaan 2: Input tc_small.json populasi 100 run 2/2



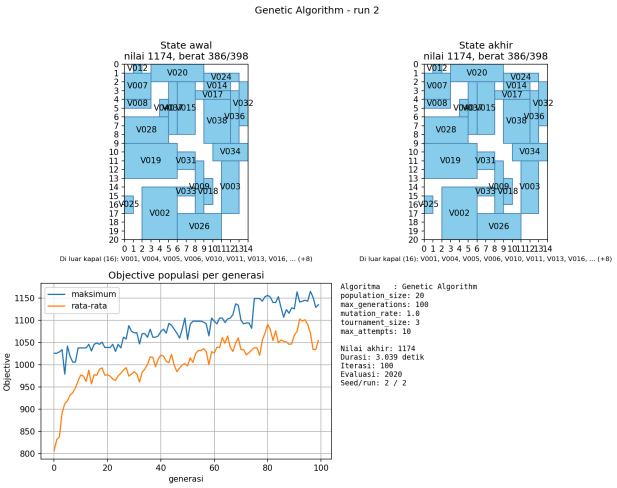
Gambar 2.4.1.4 Percobaan 2: Input tc_medium.json populasi: 20 run 2/2



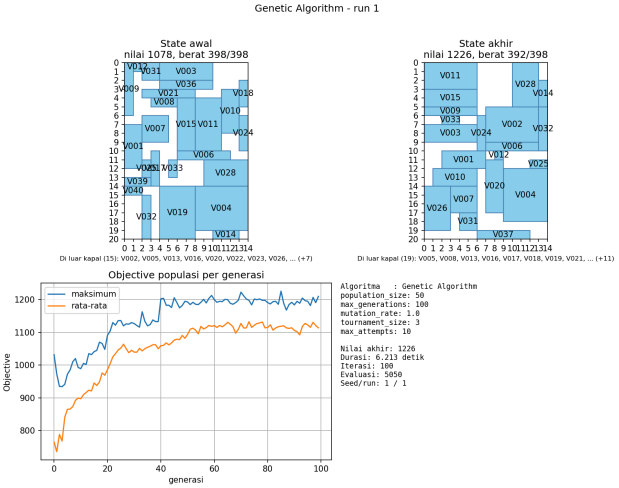
Gambar 2.4.1.5 Percobaan 2: Input tc_medium.json populasi: 50 run 1/1



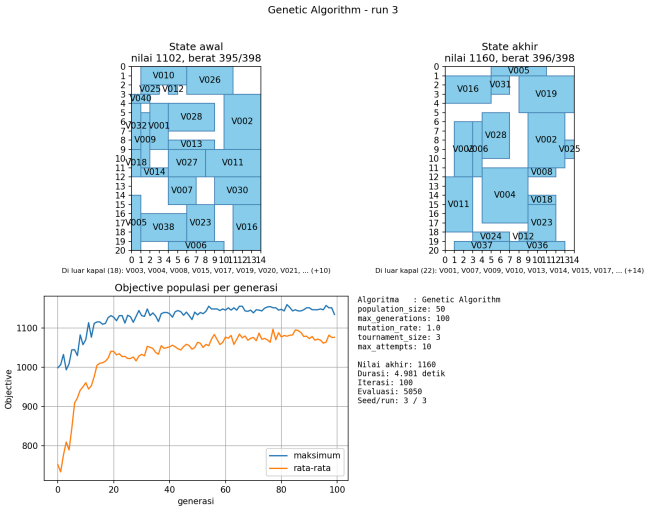
Gambar 2.4.1.6 Percobaan 2: Input tc_medium.json populasi 100 run 3/3



Gambar 2.4.1.7 Percobaan 1: Input tc_large.json iterasi 20 run 2/2



Gambar 2.4.1.8 Percobaan 1: Input tc_large.json iterasi 50run 1/1



Gambar 2.4.2.9 Percobaan 1: Input tc_large.json iterasi 100run 3/3

2.4.3 Analisis

2.4.3.1 Pengaruh ukuran populasi terhadap hasil algoritma

Peningkatan ukuran populasi memperluas keragaman genetik awal. Hal ini berdampak pada penemuan solusi dengan nilai fitness yang lebih tinggi. Pada ukuran populasi kecil (20), algoritma cenderung hanya mampu mencapai skor maksimal di kisaran 1143 hingga 1174, sedangkan penggunaan populasi 50 memberikan lonjakan skor akhir hingga mencapai 1226. Populasi besar seperti 100 juga konsisten menghasilkan fitness tinggi di angka 1214. Namun, nilai fitness tidak selalu berbanding lurus dengan peningkatan jumlah populasi karena adanya randomisasi.

2.4.3.2 Pengaruh jumlah generasi terhadap hasil algoritma

Peningkatan jumlah generasi memberikan iterasi yang lebih panjang bagi populasi untuk melakukan proses seleksi dan perbaikan solusi. Pada konfigurasi populasi

50, peningkatan generasi dari 50 ke 100 meningkatkan nilai fitness maksimum dari rentang 1149-1206 menjadi rentang 1160-1226. Generasi yang lebih banyak memberi ruang eksplorasi lebih lama bagi individu untuk mencapai global optima.

2.4.3.3 Waktu komputasi

Durasi eksekusi algoritma berbanding lurus dengan jumlah peningkatan iterasi dan populasi. Konfigurasi populasi 20 dengan 100 generasi (2.020 evaluasi) memakan waktu sekitar singkat yang relatif sebanding dengan populasi 50 dengan 50 generasi (2.550 evaluasi) di angka mulai dari 0.5 detik. Sementara itu, populasi 50 dengan 100 generasi (5.050 evaluasi) membutuhkan waktu 4,69-6,21 detik, dan populasi 100 dengan 100 generasi (10.100 evaluasi) memerlukan waktu paling lama yaitu 15,16 detik.

2.4.3.4 Konsistensi hasil

Meskipun algoritma tidak berhenti pada satu angka yang persis sama, konfigurasi ukuran populasi dan generasi mampu memandu solusi ke arah yang konsisten. Di data input medium, hasil dan rata-rata populasi naik turun secara ekstrem karena mutation_rate 100%. Konsistensi hasil dipengaruhi oleh mutation rate di tahap mutasi.

2.5 Analisis Keseluruhan dan Perbandingan

2.5.1 Dasar Perbandingan

Ketiga algoritma dibandingkan pada test case yang sama: tc_small (geladak 5×5, 6 kendaraan), tc_medium (8×12, 12 kendaraan), dan tc_large (14×20, 40 kendaraan), dengan objective dan constraint yang identik sehingga nilai akhirnya dapat dibandingkan langsung. Konfigurasi pembanding adalah yang terbaik pada tiap algoritma: Hill-Climbing dengan max_sideways = 100 (hasil untuk 200 identik), Simulated Annealing dengan suhu awal 100 dan c = 0,95, serta Genetic Algorithm dengan populasi 50 dan 100 generasi. Hill-Climbing dan Simulated Annealing dilaporkan dari tiga seed, sedangkan tabel Genetic Algorithm hanya memuat run terbaik per konfigurasi. Karena itu perbandingan lintas algoritma memakai nilai terbaik, dan rata-rata hanya dibandingkan antara Hill-Climbing dan Simulated Annealing.

2.5.2 Kedekatan terhadap Optimum Global

Optimum tc_small dihitung dengan mengenumerasi seluruh 64 himpunan bagian kendaraan dan memeriksa apakah tiap himpunan dapat ditempatkan di geladak 5×5. Hasilnya 133 (V001, V003, V004, V005). Untuk tc_medium dan tc_large dipakai batas atas dari knapsack 0/1 yang membatasi berat dan luas total. Batas ini mengabaikan geometri sehingga bersifat longgar: pada tc_small batasnya 162, 29 poin di atas optimum eksak.

Tabel 2.5.2 Nilai akhir terhadap optimum atau batas atas

Test case	Optimum / Batas atas	Hill-Climbing	Simulated Annealing	Genetic Algorithm

tc_small	133 (optimum eksak)	133 (133)	133 (133)	133
tc_medium	284 (batas atas)	281,33 (283)	281,33 (283)	283
tc_large	1.316 (batas atas)	1.234,33 (1.261)	1.070,67 (1.094)	1.226

Pada tc_small ketiga algoritma mencapai optimum eksak. Pengecualiannya Hill-Climbing tanpa sideways, yang terjebak di 90 pada dua dari tiga seed (rata-rata 104,33, atau 78,4% dari optimum). Pada tc_medium nilai terbaik semuanya 283, yaitu 99,65% dari batas atas. Selisih satu poin itu kemungkinan berasal dari kelonggaran batas atas, tetapi tidak dapat dipastikan tanpa optimum eksak. Pada tc_large, Hill-Climbing terbaik mencapai 95,8% dari batas atas (rata-rata 93,8%), Genetic Algorithm 93,2% pada populasi 50 dan 94,8% pada populasi 100, dan Simulated Annealing hanya 83,1% (rata-rata 81,4%). Karena batas atas adalah relaksasi, jarak sebenarnya ke optimum paling banyak 4,2% untuk Hill-Climbing terbaik dan 16,9% untuk Simulated Annealing terbaik.

2.5.3 Perbandingan Hasil antaralgoritma

Pada tc_small dan tc_medium nilai terbaik ketiga algoritma sama, sehingga keduanya belum membedakan kualitas solusi. Pada Genetic Algorithm tc_medium, populasi 20 (seed 2) dan populasi 100 (seed 3) bahkan sudah memuat state bernilai 283 sejak populasi awal, jadi nilai itu sebagian berasal dari inisialisasi acak. Pembeda baru muncul di tc_large, dengan urutan Hill-Climbing, Genetic Algorithm, lalu Simulated Annealing.

Hill-Climbing unggul karena setiap iterasinya mengevaluasi seluruh tetangga: tanpa sideways, seed 1 sudah mencapai 1.252 dalam 10 iterasi. Sideways 100 menaikkan rata-rata dari 1.220,67 menjadi 1.234,33 (sekitar 1,1%), dan sideways 200 tidak menambah apa pun. Seluruh 27 run berhenti karena jatah sideways habis, tidak ada yang di puncak lokal murni, sehingga pencarian berhenti di plateau yang perbaikannya membutuhkan perubahan beberapa kendaraan sekaligus.

Simulated Annealing paling rendah terutama karena anggaran iterasinya kecil. Suhu yang dimulai dari 100 sudah di bawah 0,05 setelah 73, 98, dan 149 iterasi untuk c = 0,90, 0,925, dan 0,95, jauh sebelum batas 5.000 iterasi. Menggandakan iterasi dari c = 0,90 ke 0,95 menaikkan rata-rata 30 poin (1.040,67 menjadi 1.070,67), yang mendukung penjelasan ini.

Genetic Algorithm berada di antara keduanya. Pada populasi 50, menaikkan generasi dari 50 ke 100 menaikkan nilai terbaik dari 1.206 ke 1.226, dan 200 generasi tidak menambahnya. Populasi 100 mencapai 1.247, atau 98,9% dari nilai terbaik Hill-Climbing. Perlu dicatat bahwa titik awalnya tidak setara: populasi awal terbaik Genetic Algorithm di tc_large sudah 1.078 (naik +148 menjadi 1.226), sedangkan Hill-Climbing seed 1 mulai dari satu state acak bernilai 910 (naik +347 menjadi 1.257).

2.5.4 Perbandingan Durasi dan Jumlah Evaluasi

Tabel 2.5.4 Durasi rata-rata dan beban evaluasi pada konfigurasi pembanding

Test case	Hill-Climbing	Simulated Annealing	Genetic Algorithm

tc_small	0,073 s ± 2.016 evaluasi	0,212 s 149 iterasi	0,425 s 5.050 evaluasi
tc_medium	1,19 s ± 9.980 evaluasi	5,21 s 149 iterasi	0,819 s 5.050 evaluasi
tc_large	91,9 s ± 87.533 evaluasi	352,0 s 149 iterasi	6,21 s 5.050 evaluasi

Iterasi tidak setara antaralgoritma. Satu iterasi Hill-Climbing mengevaluasi seluruh tetangga (sekitar 470 evaluasi pada tc_large), satu iterasi Simulated Annealing hanya satu tetangga, dan satu generasi Genetic Algorithm satu evaluasi per individu. Meski evaluasinya paling sedikit, Simulated Annealing justru paling lama: 352 detik untuk 149 iterasi di tc_large (sekitar 2,4 detik per iterasi), sedangkan satu iterasi Hill-Climbing hanya 0,45 sampai 0,56 detik. Sesuai Bagian 2.3.1.4, penyebabnya adalah random_neighbor() yang masih memanggil all_neighbors(), sehingga durasinya mencerminkan implementasi, bukan sifat algoritmanya.

Durasi naik tajam seiring ukuran test case: Hill-Climbing sekitar 16 kali lalu 77 kali lipat (0,073 s, 1,19 s, 91,9 s), Genetic Algorithm dari 0,425 s ke 6,21 s. Pada Genetic Algorithm durasi mengikuti beban evaluasi (di tc_large, populasi 20 ke 100 menaikkan durasi dari 3,04 s ke 14,83 s). Biaya sideways pada Hill-Climbing tidak sebanding dengan hasilnya: di tc_large durasi naik dari 5,7 s ke 91,9 s untuk tambahan

nilai sekitar 1,1%. Dari sisi efisiensi, Genetic Algorithm terbaik di tc_large: 1.226 dalam 6,21 detik, sekitar 99,3% dari rata-rata Hill-Climbing dengan 6,8% dari waktunya.

2.5.5 Konsistensi Hasil antar-run

Tabel 2.5.5 Sebaran nilai akhir pada tiga seed: minimum sampai maksimum

Test case	Hill-Climbing sideways 100	Simulated Annealing c = 0,90	Simulated Annealing c = 0,925	Simulated Annealing c = 0,95
tc_small	133–133 (0,00)	133–133 (0)	133–133 (0)	133–133 (0)
tc_medium	278–283 (2,89)	252–283 (17,9)	283–283 (0)	283–283 (0)
tc_large	1.185–1.261 (42,77)	1.026–1.058 (16,17)	1.006–1.061 (30,92)	1.048–1.094 (23,01)

Pada tc_small semua algoritma konsisten di 133, kecuali Hill-Climbing tanpa sideways (simpangan baku 24,83) yang bergantung pada state awal. Pada tc_medium dan tc_large, seed 2 Hill-Climbing selalu terendah (278 dan 1.185). Di tc_large itu sejalan dengan state awalnya yang paling rendah (873), tetapi di tc_medium state awal seed 2 (235) lebih tinggi daripada seed 1 (192), jadi hasil ditentukan juga oleh bentuk lembah tempat pencarian dimulai. Sideways menurunkan simpangan baku tc_large dari 52,55 ke 42,77.

Pada Simulated Annealing, c = 0,90 sekali turun ke 252 di tc_medium, sedangkan c = 0,925 dan 0,95 selalu 283. Di tc_large simpangan bakunya (16 sampai 31) lebih kecil daripada Hill-Climbing,

tetapi itu konsistensi di level rendah: nilai terbaik Simulated Annealing (1.094) masih di bawah nilai terburuk Hill-Climbing (1.185). Untuk Genetic Algorithm, konsistensi hanya dapat ditaksir dari rentang pada Bagian 2.4.3: 1.143 sampai 1.174 pada populasi 20, dan 1.160 sampai 1.226 pada populasi 50 dengan 100 generasi, rentang 66 poin yang sebanding dengan Hill-Climbing (76 poin).

2.5.6 Ringkasan dan Keterbatasan

Pada tc_small dan tc_medium ketiga algoritma mencapai nilai terbaik yang sama (133 dan 283). Pada tc_large, Hill-Climbing memberi nilai tertinggi dengan evaluasi terbanyak, Genetic Algorithm memberi nilai hampir setara dengan durasi sekitar lima belas kali lebih singkat, dan Simulated Annealing memberi nilai terendah dengan durasi terlama pada konfigurasi yang diuji.

Kesimpulan ini punya empat batasan. Batas atas adalah relaksasi, sehingga persentase kedekatan adalah batas bawah. Nilai Genetic Algorithm adalah run terbaik, bukan rata-rata. Tiap ukuran hanya satu test case dengan tiga seed, sehingga simpangan baku sensitif terhadap satu run. Simulated Annealing tidak diberi anggaran iterasi sebanding dengan dua algoritma lain (kriteria berhenti $T < 0{,}05$ dan random_neighbor() yang mahal), sehingga kekalahannya berlaku untuk konfigurasi dan implementasi ini, bukan untuk Simulated Annealing secara umum.

Bab III: Kesimpulan dan Saran

3.1 Kesimpulan

a. Simulated Annealing

Simulated Annealing terbukti efektif lolos dari local optima karena adanya mekanisme probabilitas penerimaan state yang lebih buruk menggunakan penurunan suhu secara bertahap. Keberhasilan dan tingkat konsistensi algoritma ini dipengaruhi oleh cooling rate (α). Niilai α yang lebih tinggi (seperti 0.95) memberikan kesempatan eksplorasi lebih banyak sehingga menghasilkan nilai objective yang lebih optimal pada test case besar. Namun, downsidenya adalah algoritmanya memerlukan durasi komputasi yang lebih lama akibat peningkatan jumlah iterasi. Durasi eksekusi meningkat secara seiring membesarnya ukuran test case karena tingginya biaya pemeriksaan validitas state pada setiap langkah.

b. Hill Climbing

Algoritma Hill Climbing menawarkan efisiensi waktu komputasi yang cepat pada skala persoalan kecil hingga menengah. Namun, algoritma ini rentan terjebak pada area datar (stuck). Penerapan sideways move terbukti secara signifikan meningkatkan peluang algoritma keluar dari plateau dan meraih nilai objective yang lebih tinggi dibandingkan hill climbing murni. Meskipun demikian, penambahan jatah sideways melebihi batas efektif (contoh: dari 100 ke 200) tidak memberikan peningkatan nilai objective lebih lanjut dan hanya menambah beban durasi eksekusi.

c. Genetic

Genetic Algorithm menunjukkan performa pencarian yang baik untuk test case besar. Kombinasi ukuran populasi dan jumlah generasi yang memadai memperluas variasi genetik dan daya eksplorasi ruang pencarian. Hal ini menyebabkan algoritma mampu menghasilkan nilai fitness tinggi. Performa tinggi ini dibarengi dengan beban komputasi dan waktu eksekusi yang besar akibat tingginya jumlah evaluasi fitness, operasi mating, dan mutasi pada setiap generasi. Namun, konsistensi hasil akhir dipengaruhi oleh rate mutasi (rate mutasi makin besar -> tingkat variansi makin banyak)

3.2 Saran

1. Perbaiki fungsi random_neighbor supaya algoritmanya jadi lebih efisien. Fungsi random_neighbor yang kurang bagus bisa menyebabkan cost membengkak saat algoritma dijalankan.
2. Buat direktori ouput yang lebih baik dan terorganisasi (pisahkan png dan json, pisahkan juga tiap-tiap folder output algoritma instead of digabungin di satu tempat.
3. Bikin pseudocode yang mateng dulu baru mulai nulis codenya supaya tidak ada perbaikan lebih lanjut saat teman-teman lain sudah pakai fungsinya (maaf teman-teman).

Pembagian Tugas

Nama : Tsabita Khoirunnisa

NIM : 18224006

No	Tugas	Deskripsi
1	Inisialisasi pembagian tugas	Menyusun dokumen rencana pengerjaan tugas dan pembagian peran
2	Mengerjakan core.py	Menyusun seluruh fungsi dalam core.py dan file testnya di test_core.py
3	Mengerjakan hill_climbing.py	Menyusun seluruh fungsi yang berkaitan dengan algoritma hill climbing dan file testnya di test_hill_climbing.py
4	Mengerjakan runner.py, generator_testcase.py, dan visualisasi.py	Memperbaiki dan menambahkan isi file runner.py, generator_testcase.py, dan visualisasi.py yang sudah dikerjakan anggota lain
5	Penulisan core.py, visualisasi.py, dan hill_climbing.py pada laporan	Menulis dan menjelaskan seluruh fungsi dalam core.py, visualisasi.py, dan hill_climbing.py

Nama : Wa Ode Amerta Lambelu Jamaluddin

NIM : 18224034

No	Tugas	Deskripsi
1	Mengerjakan problem.py	Mengerjakan file problem.py sebagai bagian dari shared function yang dipakai di seluruh algoritma
2	Mengerjakan genetic_algorithm.py	Menuliskan code genetic_algorithm.py
3	Mengerjakan testcase_generator.py	Menuliskan code generator_testcase.py
4	Menuliskan algoritma genetik di laporan	Menyediakan state awal dan akhir dari eksperimen algoritma genetik dan melakukan analisis dalam laporan terhadap algoritma genetik.
5	Menuliskan problem.py dan generator_testcase.py di laporan	Menuliskan dan menjelaskan singkat fungsi-fungsi yang ada di problem.py dan generator_testcase.py

Nama : Melvin Irsyad Kuncoro

NIM : 18224048

No	Tugas	Deskripsi
1	Mengerjakan neighbors.py	Menyusun seluruh fungsi dalam neighbors.py

2	Mengerjakan simulated_annealing.py	Menyusun seluruh fungsi yang berkaitan dengan algoritma simulated annealing
3	Inisialisasi pengerjaan visualisasi.py	Mengawali pembuatan visualisasi.py untuk menggambarkan state dan juga grafik tren. Visualisasi.py kemudian disempurnakan oleh anggota lain.
4	Penulisan neighbors.py pada laporan	Menulis dan menjelaskan seluruh fungsi dalam neighbors.py
5	Penulisan runner.py pada laporan	Menulis dan menjelaskan fungsi-fungsi yang terdapat dalam runner.py
6.	Penulisan simulated_annealing.py, seluruh eksperimennya, dan analisis pada laporan	Menuliskan dan menjelaskan source code SA, eksperimen yang dilakukan, dan analisis pada laporan.
7.	Membantu formatting dokumen dan menambahkan README.md	Membenarkan penggunaan header pada dokumen, merapikan daftar isi, tabel, dan gambar. Menulis README.md pada repository

Referensi

Collins, E. R. (2024, March 11). 5 best ways to detect rectangle overlap in Python. Finxter. https://blog.finxter.com/5-best-ways-to-detect-rectangle-overlap-in-python/

GeeksforGeeks. (2024, September 12). *What is simulated annealing*. GeeksforGeeks. https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/

Khodra, M. L. (2021). Beyond classical search: Hill-climbing search [Video]. YouTube. https://www.youtube.com/watch?v=L0b0CN3Fds8

listx. (n.d.). Rectangle overlap (intersection). listx.srht.site. https://listx.srht.site/codex/problem/rectangle_overlap/README.html

Python Software Foundation. (n.d.-a). copy: Shallow and deep copy operations. Python 3.11 documentation. https://docs.python.org/3.11/library/copy.html

Python Software Foundation. (n.d.-b). unittest: Unit testing framework. Python 3 documentation. https://docs.python.org/3/library/unittest.html

villares. (2022, May 22). irregular_grid.pyde [Source code]. GitHub Gist. https://gist.github.com/villares/f2072b3651a4a70d601590957fe83677

Referensi repository:

https://github.com/eriklindernoren/ML-From-Scratch.git

https://github.com/Am1rMahd1/Genetic-algorithm-for-0-1-Knapsack-Problem.git

https://github.com/2black0/Simple-Genetic-Algorithm-in-Python.git

Lampiran Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Tsabita Khoirunnisa (18224006), Wa Ode Amerta Lambelu Jamaluddin (18224034), Melvin Irsyad Kuncoro (18224048)

Fakultas/Sekolah : STEI

Kode Mata Kuliah : IF3070

Nama Mata Kuliah : Dasar Inteligensi Artifisial

Judul Tugas/Proposal : Tugas Besar 1

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

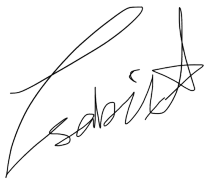
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	AI hanya digunakan untuk merapikan penyusunan paragraf serta alur penulisan yang lebih baik.
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	AI hanya digunakan untuk merapikan penyusunan ataupun menjadi referensi penulisan.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI digunakan untuk berdiskusi terkait fungsi atau algoritma yang sedang dibangun ketika mengalami

		kebingungan. AI juga digunakan untuk mencari solusi dari bug yang sedang dialami jika tidak kunjung menemukan solusi.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	Tidak digunakan, grafik pada tugas ini dibuat dengan library python matplotlib
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Tsabita Khoirunnisa

NIM 18224006



Wa Ode Amerta Lambelu J

NIM 18224034



Melvin Irsyad Kuncoro

NIM 1822048

Lampiran Lainnya